

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION COMMUNICATION TECHNOLOGY



SOICT

PROJECT REPORT:

Building a Movie Data Analysis Pipeline

Supervised by:

Ph.D. Tran Viet Trung

Presented by:

Au Trung Phong - 20225455

Nguyen Nam Khanh - 20225448

Vu Duc Minh - 20225514

Tran Sy Minh Quan - 20225521

Bui Van Huy - 20225497

Hanoi - Vietnam
2025

Contents

1	Problem Definition	2
1.1	Selected Problem	2
1.2	Analysis of Suitability for Big Data	3
1.3	Scope and Limitations	4
2	Architecture and Design	5
2.1	Overall Architecture	5
2.2	Detailed Components and Roles	7
2.2.1	Speed Layer	7
2.2.2	Batch Layer	7
2.2.3	Serving Layer	7
2.3	Data Flow and Interactions	8
2.3.1	Speed Layer: Real-Time Reddit Data Flow	8
2.3.2	Batch Layer: Historical TMDb Data Flow	10
2.3.3	Serving Layer: Real-Time Query Interface	12
3	Implementation Details	16
3.1	Source Code Documentation	16
3.1.1	Batch Layer - Historical Data Processing	17
3.1.2	Speed Layer - Real-Time Data Processing	24
3.1.3	Serving Layer - Query-Time Merge and Analytics	31
3.2	Environment Configuration	45
3.2.1	Batch Layer Container Configuration	45
3.2.2	Speed Layer Container Configuration	46
3.2.3	Serving Layer Container Configuration	46
3.2.4	Environment Variable Configuration	47
3.3	Deployment Strategy	49
3.3.1	Approach: Automated Conversion with Iterative Debugging	49
3.3.2	Critical Blocker and Deployment Status	50
3.4	Monitoring Setup	50
3.4.1	Infrastructure Metrics Collection	50
3.4.2	Alert Rules and Thresholds	51
3.4.3	Grafana Visualization Architecture	51
3.4.4	Health Check and Monitoring Endpoints	54
3.4.5	Monitoring Access and Integration	55
4	Lessons Learned	56

4.1	Lesson 1: Strategic Planning	56
4.1.1	Problem Description	56
4.1.2	Approaches Tried	56
4.1.3	Final Solution	57
4.1.4	Key Takeaways	57
4.2	Lesson 2: API Schema Consistency in Serving Layer Integration	57
4.2.1	Problem Description	57
4.2.2	Approaches Tried	57
4.2.3	Final Solution	58
4.2.4	Key Takeaways	58
4.3	Lesson 3: Dependency Validation and Configuration Management	58
4.3.1	Problem Description	58
4.3.2	Proposed Approaches	58
4.3.3	Future Recommendations	59

1 Problem Definition

1.1 Selected Problem

In the modern film and entertainment industry, audience perception and public discourse increasingly shape the commercial and reputational outcomes of movies. Online social platforms allow viewers to react immediately, creating fast-evolving engagement signals that influence marketing effectiveness, brand reputation, and content visibility.

Based on the business requirements outlined in this project, three core problems are identified.

Problem 1: PR crisis detection and sentiment monitoring remain insufficiently responsive. Negative audience reactions often surface on social platforms well before traditional metrics such as ratings or box office performance reflect them. Without systematic comparison against historical sentiment norms, temporary fluctuations may be mistaken for crises, or genuine crises may be detected too late. Entertainment companies therefore require a mechanism to identify statistically significant drops in audience sentiment relative to historical baselines using standard deviation (σ) analysis, enabling rapid intervention within critical response windows. For example, a sudden decline in Reddit sentiment for *Dune: Part Two* to $+0.3$, compared to a historical science-fiction baseline of $+0.65$ (with standard deviation $\sigma = 0.15$), yields a deviation of -2.33σ , exceeding the -2.0σ warning threshold and indicating a meaningful reputational issue rather than normal variance.

Problem 2: Identifying genuinely viral content for marketing amplification is challenging. High engagement volume alone does not necessarily imply virality, as discussion levels vary widely across genres, budgets, and release periods. Marketing teams must distinguish organic viral moments from expected engagement fluctuations in order to act within the short 24–48 hour window during which amplification is most effective. This requires calculating a viral coefficient by comparing a composite

engagement metric against genre-specific popularity baselines to determine whether current attention levels represent an exceptional event. The system computes a weighted viral score combining upvote velocity (50%), comment velocity (30%), and award velocity (20%) from Reddit discussions, then divides this by the average TMDb popularity score for the corresponding genre. For instance, detecting discussion around *The Creator* with a viral score of 5.2 compared against a genre baseline popularity of 42.0 yields a viral coefficient of 0.124, which exceeds the 0.10 threshold calibrated for viral status, signaling a genuine viral opportunity worthy of immediate marketing action.

Problem 3: Content recommendation and prioritization lack real-time awareness. High engagement volume alone does not necessarily imply virality, as discussion levels vary widely across genres, budgets, and release periods. Marketing teams must distinguish organic viral moments from expected engagement fluctuations in order to act within the short 24–48 hour window during which amplification is most effective. This requires calculating a viral coefficient by comparing a composite engagement metric against genre-specific popularity baselines to determine whether current attention levels represent an exceptional event. The system computes a weighted viral score combining upvote velocity (50%), comment velocity (30%), and award velocity (20%) from Reddit discussions, then divides this by the average TMDb popularity score for the corresponding genre. For instance, detecting discussion around *The Creator* with a viral score of 5.2 compared against a genre baseline popularity of 42.0 yields a viral coefficient of 0.124, which exceeds the 0.10 threshold calibrated for viral status, signaling a genuine viral opportunity worthy of immediate marketing action.

In summary, the central problem addressed in this project is: **how to combine real-time social engagement signals with historical movie intelligence to support rapid crisis detection, reliable viral identification, and dynamic content prioritization in the film and entertainment industry.**

1.2 Analysis of Suitability for Big Data

The three problems identified above require continuous processing of fast-changing social signals, systematic comparison against historical context, and scalable analytical workflows. These requirements align naturally with Big Data technologies, which are designed to handle streaming data, large-scale historical aggregation, and complex query-time analytics.

Suitability for Problem 1: PR Crisis Detection and Sentiment Monitoring. PR crises emerge rapidly and demand immediate analytical response. Big Data streaming frameworks enable continuous ingestion and near real-time processing of social media discussions, allowing sentiment trends to be evaluated as new data arrives. At the same time, Big Data batch processing supports the construction of robust historical sentiment baselines from long-term movie review data. By comparing real-time sentiment against these baselines, the system can identify statistically significant negative deviations, distinguishing genuine crises from short-lived noise. This capability directly supports early warning mechanisms that are essential for timely PR intervention.

Suitability for Problem 2: Viral Content Identification for Marketing Amplification. Detecting viral behavior requires both velocity awareness and historical context. Big Data batch analytics makes it pos-

sible to compute historical engagement thresholds across different genres, budgets, and release periods, providing a quantitative definition of what constitutes “viral” behavior. In parallel, Big Data streaming processing captures real-time engagement velocity metrics such as upvote rate and comment growth. By evaluating current activity against historical thresholds, Big Data systems enable reliable identification of genuine viral events within narrow time windows, allowing marketing teams to act when amplification impact is highest.

Suitability for Problem 3: Content Recommendation Optimization. Effective content prioritization depends on integrating fresh audience interest with long-term performance indicators. Big Data architectures support query-time merging of real-time social signals and historical movie intelligence, enabling recommendation systems to remain both responsive and informed. This unified analytical view allows platforms to re-rank content dynamically based on current buzz while still enforcing historical quality benchmarks. As a result, Big Data enables scalable, automated recommendation optimization that adapts continuously to evolving audience behavior.

In conclusion, Big Data technologies provide the necessary foundation to address delayed crisis detection, ambiguous viral signals, and static content prioritization. By unifying real-time analytics with historical context, Big Data transforms audience engagement data into actionable intelligence that directly supports strategic decision-making in the film and entertainment industry.

1.3 Scope and Limitations

- **Scope:**

Real-time ingestion and analysis of Reddit movie discussions: The project continuously collects Reddit posts and comments related to movies. Incoming data is processed in near real-time to extract engagement metrics such as discussion volume, upvotes, comment frequency, and sentiment scores. This enables the system to monitor current audience reactions shortly after they occur.

Batch processing of historical movie data from TMDB: Historical movie metadata and review data from TMDB are processed in batch mode. This processing computes long-term aggregates such as average sentiment, typical engagement levels, and baseline popularity metrics, which serve as reference points for interpreting real-time social activity.

Combination of real-time and historical analytics: The project integrates streaming results with batch-computed baselines to provide contextual insights. Recent Reddit activity is compared against historical patterns to determine whether observed trends represent normal behavior, abnormal sentiment shifts, or potentially viral events.

Sentiment and engagement-based movie trend detection: The system applies sentiment analysis to social media text and combines it with engagement metrics to identify trending movies. This supports use cases such as early PR crisis awareness and identification of content gaining sudden audience attention.

- **Limitations:**

Limited accuracy of sentiment analysis: Sentiment analysis is based on lightweight, rule-based or lexicon-based methods. While efficient for large-scale processing, this approach may fail to capture sarcasm, implicit sentiment, or complex linguistic nuances common in online discussions.

Restricted data sources: The project focuses primarily on Reddit for real-time social signals and TMDB for historical movie data. Other influential platforms such as Twitter, YouTube, or Instagram are not included, which limits the completeness of audience representation.

Language and regional constraints: The analysis assumes English-language content. Multilingual sentiment analysis and regional audience behavior differences are outside the scope of this project.

Business impact not quantitatively evaluated: The project demonstrates technical feasibility and analytical capabilities, but does not include a quantitative evaluation of business outcomes such as revenue improvement, marketing efficiency, or user retention.

2 Architecture and Design

2.1 Overall Architecture

The system is designed following the **Lambda Architecture** paradigm, which combines batch processing and stream processing to support both **historical accuracy** and **low-latency analytics**. This architectural model is particularly well-suited for the movie analytics domain, where large volumes of historical data coexist with high-velocity social media streams.

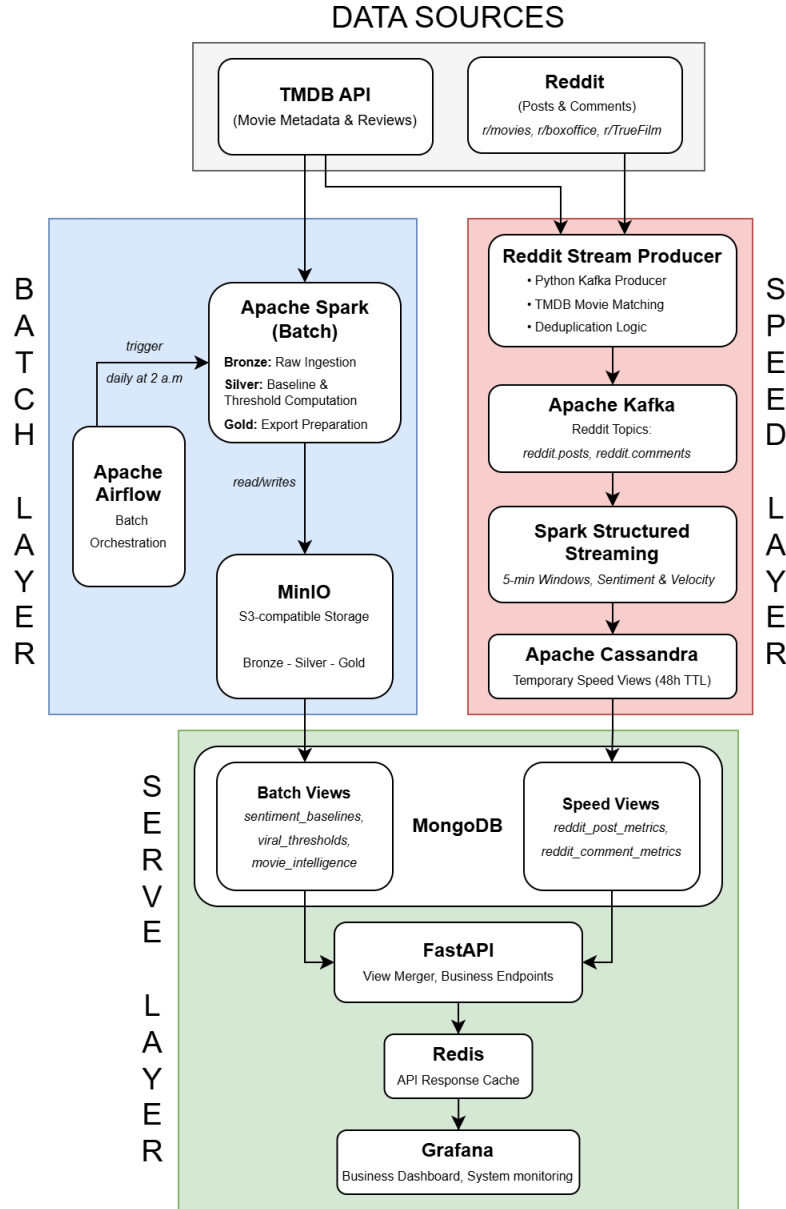


Figure 1: Overall system architecture based on the Lambda Architecture paradigm

The architecture is composed of three logical layers:

- **Batch Layer:** Processes immutable, historical data from TMDB to compute accurate long-term analytics such as sentiment baselines, genre-level trends, and viral thresholds.
- **Speed Layer:** Processes real-time Reddit discussions to provide near real-time insights into audience sentiment, engagement velocity, and emerging viral signals.
- **Serving Layer:** Exposes a unified query interface by merging batch and speed results at query time, ensuring both freshness and accuracy.

The choice of Lambda Architecture is motivated by the complementary nature of the data sources. Reddit provides fast-changing, high-velocity social engagement signals that require immediate processing, while TMDB provides stable, historical movie intelligence that serves as statistical context. By separating

these concerns while merging results at query time, the system achieves scalability, fault tolerance, and analytical consistency.

2.2 Detailed Components and Roles

The system components are categorized by their specific roles within the Lambda Architecture layers:

2.2.1 Speed Layer

This layer handles real-time data ingestion and low-latency processing.

- **Apache Kafka:** Serves as the real-time ingestion backbone for the Speed Layer. Reddit posts and comments are published to Kafka topics, decoupling data producers from downstream consumers and enabling scalable, fault-tolerant stream processing.
- **Spark Structured Streaming:** Performs continuous, low-latency processing of Reddit data. Streaming jobs compute window-based metrics such as sentiment scores, comment and upvote velocity, and viral indicators. The unified Spark API allows streaming and batch logic to share similar transformation semantics.
- **Apache Cassandra:** Acts as the primary storage for speed-layer views. Cassandra is optimized for high write throughput and horizontal scalability, making it suitable for time-series style metrics generated by streaming workloads. A time-to-live (TTL) policy is applied to automatically retain only the most recent 48 hours of data.
- **Cassandra-MongoDB Sync:** Periodically transfers speed-layer views from Cassandra to MongoDB at 5-minute intervals. This sync process ensures the serving layer has access to fresh Reddit metrics while maintaining the 48-hour data retention window through coordinated TTL policies.

2.2.2 Batch Layer

This layer manages the master dataset and computes comprehensive historical views.

- **MinIO (S3-Compatible Object Storage):** Provides distributed storage for batch data using a Bronze–Silver–Gold medallion architecture. Raw TMDb data is stored in the Bronze layer, cleaned and enriched datasets in the Silver layer, and aggregated analytical views in the Gold layer.
- **Apache Spark (Batch Processing):** Executes large-scale batch transformations on historical TMDb data. Batch jobs compute sentiment baselines, viral thresholds, and enriched movie intelligence features using distributed processing.
- **Apache Airflow:** Orchestrates batch workflows and manages job dependencies. Airflow schedules periodic recomputation of batch analytics, ensuring historical views remain consistent and reproducible.

2.2.3 Serving Layer

This layer merges real-time and historical data to respond to user queries.

- **MongoDB:** Functions as the unified serving datastore. Both batch-layer outputs and synchronized

speed-layer views are stored in MongoDB to support flexible document-based queries and efficient aggregation at query time.

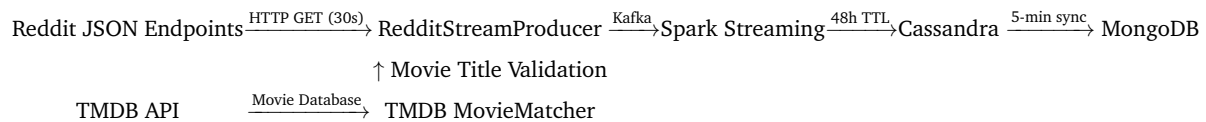
- **FastAPI:** Exposes RESTful endpoints for accessing analytics results. The API layer implements query-time merging logic, combining recent speed-layer data with historical batch-layer data to provide a consistent analytical view.
- **Redis:** Acts as an in-memory caching layer. Frequently accessed query results are cached to reduce latency and offload read pressure from MongoDB.
- **Prometheus:** Collects time-series metrics from MongoDB, Redis, and the FastAPI application. Provides the data source for Grafana dashboards and implements alerting rules for system health monitoring.
- **Grafana:** Provides real-time visualization and monitoring dashboards. Grafana is used to track system health, data freshness, and key analytical metrics across all layers.

2.3 Data Flow and Interactions

This section delineates the data propagation pathways and component interactions across the three layers of the Lambda Architecture. Each subsection details the trajectory of data through the system and the specific communication protocols employed between components.

2.3.1 Speed Layer: Real-Time Reddit Data Flow

Data Ingestion Flow:



Component Interactions:

1. Reddit JSON Endpoints → Kafka Producer:

The producer polls Reddit’s JSON endpoints every 30s to fetch up to 100 posts and comments from targeted subreddits (`r/movies`, `r/boxoffice`, `r/TrueFilm`), maintaining a 2s delay to respect rate limits. Extracted movie titles undergo a two-stage fuzzy matching process via `MovieMatcher`, which maintains a cached TMDB movie database (populated from the batch layer’s TMDB ingestion), applying a ≥ 0.75 similarity threshold and a ≥ 0.8 validation against this cache, before being serialized with gzip compression. Finally, these validated entities are published to the `reddit.posts` and `reddit.comments` Kafka topics, which are configured with 3 partitions and 48-hour retention.

2. Kafka → Spark Streaming:

Apache Spark consumes data from both topics using the Structured Streaming API, discretizing the continuous stream into 5-minute tumbling windows based on the `created_utc` timestamp with 30-second watermarking for late arrivals. Within each window, the system applies a User Defined

Function (UDF) utilizing VADER (Valence Aware Dictionary and sEntiment Reasoner) to compute sentiment polarity scores ranging from -1.0 (negative) to +1.0 (positive) for post titles combined with selftext and comment bodies. Concurrently, velocity-based viral scores are calculated using the weighted formula:

$$\text{viral_score} = 0.5 \times \frac{\text{total_upvotes}}{300} + 0.3 \times \frac{\text{total_comments}}{300} + 0.2 \times \frac{\text{total_awards}}{300}$$

where velocities are normalized to per-second rates over the 300-second (5-minute) window.

3. Spark Streaming → Cassandra:

Processed metrics are aggregated per movie within each time window and persisted to two Cassandra tables: `reddit_post_metrics` and `reddit_comment_metrics`. The storage schema utilizes a composite primary key structure `PRIMARY KEY ((movie_title, hour), window_start)` where the partition key combines movie title with hourly granularity for efficient data distribution, and `window_start` serves as the clustering key sorted in descending order to optimize time-range retrieval. A 48-hour Time-To-Live (TTL) policy is enforced at the table level via the `default_time_to_live` parameter, ensuring automatic expiration of stale data.

4. Cassandra → MongoDB Sync:

A dedicated sync connector transfers speed-layer views from Cassandra to MongoDB's `speed_views` collection at 5-minute intervals. This synchronization process creates indexed MongoDB documents with a coordinated TTL policy (`ttl_expires_at` field) to maintain the 48-hour retention window, enabling the serving layer to perform unified queries across batch and speed data without direct Cassandra access.

Data Attributes at Each Stage. The data undergoes progressive transformation across the speed layer pipeline. At the Kafka layer, raw posts and comments are stored with their associated engagement metrics including upvotes (score), Reddit awards (gold/platinum), comment counts, and creation timestamps. The Spark streaming stage produces window-aggregated outputs including VADER compound sentiment scores, velocity metrics computed as per-second rates over 5-minute windows, and weighted viral scores combining engagement signals. Finally, Cassandra storage maintains time-series views partitioned by `(movie_title, hour)` and clustered by `window_start` in descending order, enabling efficient temporal range queries while automatically expiring data older than 48 hours.

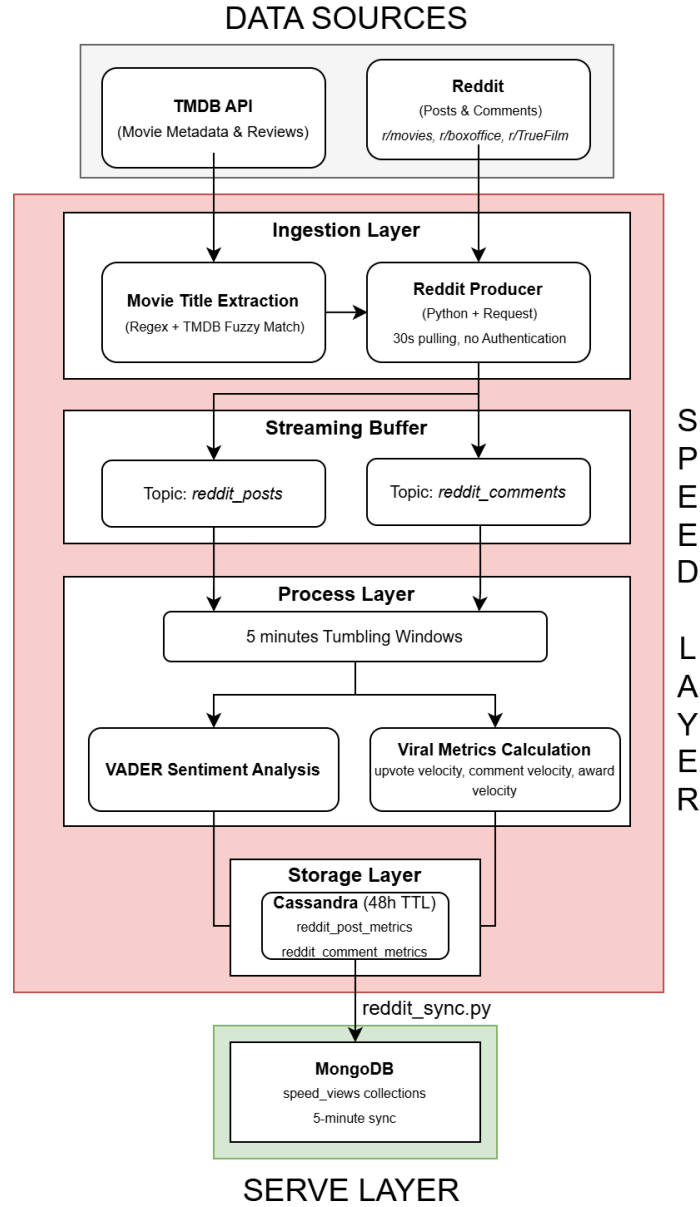


Figure 2: Speed Layer Data Flow and Interactions

2.3.2 Batch Layer: Historical TMDB Data Flow

Data Processing Flow:

TMDB API $\xrightarrow{\text{Airflow DAG}}$ Bronze (MinIO) $\xrightarrow{\text{Spark}}$ Silver (MinIO) $\xrightarrow{\text{Spark}}$ Gold (MinIO) $\xrightarrow{\text{PyMongo}}$ MongoDB

Component Interactions:

1. TMDB API $\rightarrow$ Bronze Layer:

The workflow is orchestrated by the `tmdb_baseline_pipeline` DAG, which executes daily at 02:00 UTC. This process triggers the `bronze_ingest.py` Spark job to fetch movie metadata from categories such as `popular` and `top_rated`. The ingestion adheres to a strict rate limit of 4

requests per second and deduplicates movies appearing across multiple categories before persisting the raw data as immutable Parquet files in the `s3a://bronze/tmdb_movies/` directory of MinIO. Review data is collected from the top 50 most popular movies for baseline sentiment calculation.

2. Bronze → Silver Layer:

The `silver_transform.py` job processes raw Bronze data to generate three distinct enriched datasets: `sentiment_baselines`, `viral_thresholds`, and `movie_intelligence`. During this phase, simplified sentiment analysis is applied to historical reviews using a VADER-inspired word-list approach (positive word counting minus negative word counting), and records are enriched with derived attributes including franchise detection via `belongs_to_collection` field, budget tier classification (indie: <\$20M, mid: \$20M-\$100M, blockbuster: >\$100M), and seasonal release patterns. The transformed results are stored in separate Silver layer directories.

3. Silver → Gold Layer:

The `gold_aggregate.py` job aggregates the three Silver datasets into production-ready views without merging them. This transformation involves appending temporal metadata such as `batch_run_timestamp`, `aggregation_granularity`, and `data_period_start/end` boundaries, as well as normalizing schemas by converting genre arrays to comma-separated string formats. The finalized datasets are written to separate `s3a://gold/{dataset_name}/` directories, maintaining clear separation for the three business goals.

4. Gold → MongoDB:

The final export phase is handled by `export_to_mongo.py`, which reads the three Gold Parquet files and populates the serving database with dedicated collections. The process initializes three specific MongoDB collections (`sentiment_baselines`, `viral_thresholds`, `movie_intelligence`) with optimized compound indexes, such as `(genre, year)` for sentiment baselines and `(genre, budget_tier, season)` for viral thresholds. Then, it executes bulk upsert operations via `ReplaceOne` in batches of 1000 documents to ensure data consistency and minimize lock contention.

Data Transformations Across Layers. The batch processing pipeline implements a multi-stage transformation architecture following the medallion pattern. The Bronze layer preserves raw TMDB JSON responses in immutable Parquet format with deduplication across categories, ensuring complete data lineage and reproducibility. Subsequently, the Silver layer enriches this Parquet data with computed features including franchise affiliations detected via collection membership, budget tier classifications based on production budget thresholds, and simplified sentiment scores derived from word-list analysis of review text from top movies. The Gold layer then produces three production-ready datasets with normalized schemas and comprehensive batch metadata encompassing execution timestamps and data period boundaries. Finally, MongoDB collections store these refined datasets in separate collections with compound indexes strategically designed to minimize query latency for serving layer operations, with each collection optimized for its specific business goal.

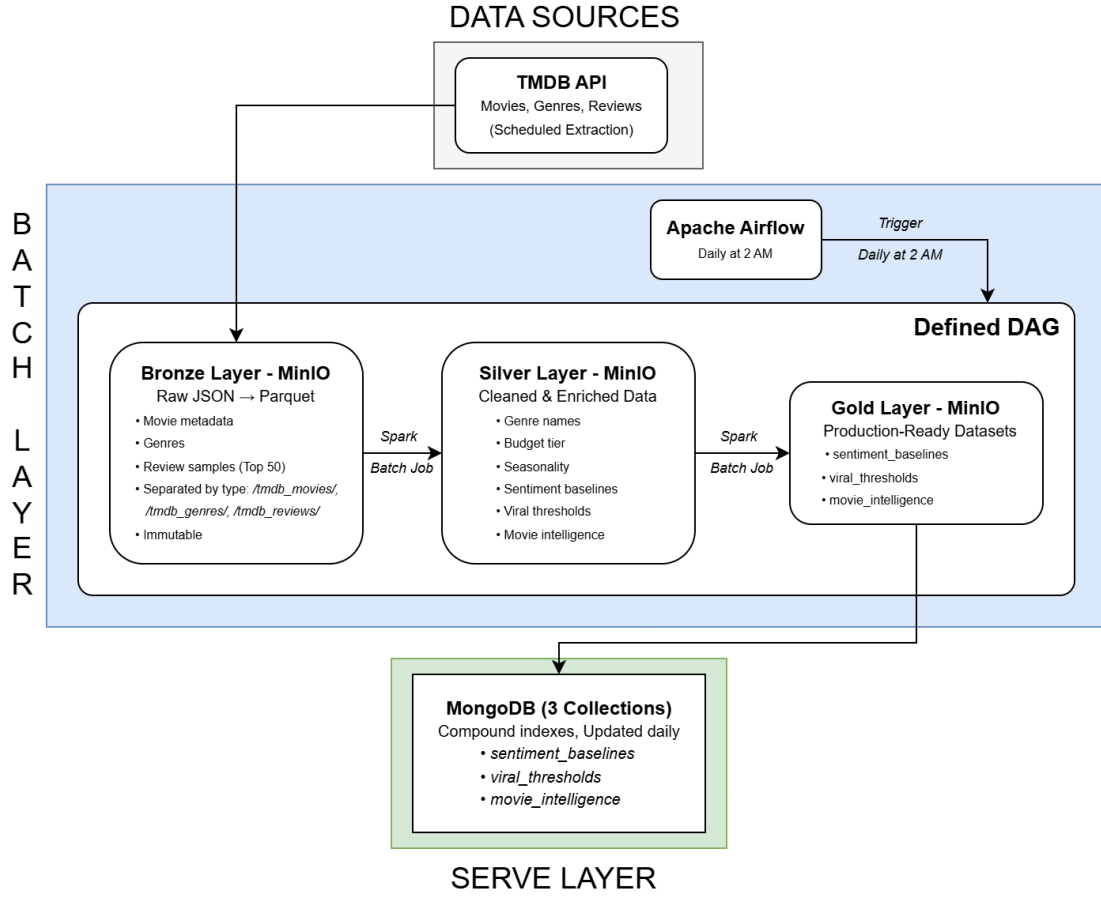


Figure 3: Batch Layer Data Flow and Interactions

2.3.3 Serving Layer: Real-Time Query Interface

Data Flow:

MongoDB $\xrightarrow{\text{Query}}$ FastAPI (8000) $\xrightarrow{\text{Cache}}$ Redis $\xrightarrow{\text{Metrics}}$ Prometheus $\xrightarrow{\text{Visualize}}$ Grafana

Component Interactions:

1. **MongoDB → FastAPI:** FastAPI REST API (port 8000) queries six MongoDB collections: three batch views (`sentiment_baselines`, `viral_thresholds`, `movie_intelligence`) updated daily, and one speed view (`speed_views`) synchronized from Cassandra with 48-hour TTL. Three routers implement business logic: (1) Crisis Detection computes deviation $\sigma = (S_{\text{current}} - S_{\text{baseline}}) / \sigma_{\text{baseline}}$ with severity thresholds (critical < -70 , high < -30 , warning < -25), (2) Viral Detection calculates $V = \sum \text{viral_score} / \text{avg_popularity}$ with status classification (viral ≥ 0.10 , trending ≥ 0.05 , growing ≥ 0.01), (3) Recommendations computes dual-success score $(0.6 \times \text{Reddit}) + (0.4 \times \text{TMDB})$ from normalized components. Connection pooling: 10-100 connections, 5-second timeout.
2. **FastAPI → Redis:** Redis cache layer provisioned (30-minute TTL, LRU eviction) but not actively used. API queries MongoDB directly using compound indexes for sub-50ms latencies.

3. **API → Prometheus:** Metrics endpoint exposes infrastructure (MongoDB/Redis exporters on ports 9216/9121), business (alerts, detections, requests), and performance metrics. 15-second scrape intervals, 15-day retention.
4. **API → Infinity:** Infinity datasource queries REST API endpoints directly for real-time data retrieval, enabling Grafana panels to display current system state, crisis alerts, viral movies, and recommendation scores without pre-aggregation.
5. **Prometheus/Infinity → Grafana:** Four dashboards combine Prometheus (time-series metrics) and Infinity (live JSON queries) datasources: System Health, Crisis Detection, Viral Content, Recommendation Performance. Auto-refresh: 30s-5m intervals.

Business Logic Formulas.

Goal #1: Crisis Detection.

Intuition: Detect PR crises by comparing current Reddit sentiment against historical TMDB baselines. Large negative deviations indicate reputation damage requiring immediate response.

Current sentiment calculation:

$$S_{current} = \begin{cases} \frac{1}{n} \sum_{i=1}^n S_{speed,i} & \text{if speed data exists (48h)} \\ S_{batch} & \text{otherwise} \end{cases} \quad (1)$$

where $S_{speed,i}$ is sentiment from the i -th 5-minute window, n is total windows in 48 hours, and S_{batch} is historical TMDB sentiment. This prioritizes fresh Reddit discussions when available, falling back to stable historical data otherwise.

Statistical deviation:

$$\sigma_{deviation} = \frac{S_{current} - \mu_{baseline}}{\sigma_{baseline}} \quad (2)$$

where $\mu_{baseline}$ and $\sigma_{baseline}$ come from hierarchical baselines (franchise > genre > year). This standardizes deviations across different movie contexts—a -2.0 sentiment drop means different things for comedy vs. drama.

Severity classification:

$$\text{severity} = \begin{cases} \text{critical} & \text{if } \sigma_{deviation} < -70 \\ \text{high} & \text{if } \sigma_{deviation} < -30 \\ \text{warning} & \text{if } \sigma_{deviation} < -25 \\ \text{normal} & \text{otherwise} \end{cases} \quad (3)$$

Thresholds calibrated to real data: critical captures bottom 1% (severe crises), high captures bottom 5% (significant issues), warning captures bottom 15% (early signals).

Goal #2: Viral Detection.

Intuition: Identify content gaining disproportionate Reddit traction relative to its TMDB popularity. High viral coefficients reveal organic buzz that marketing can amplify.

Viral coefficient:

$$V = \frac{\sum_{w \in W_{48h}} \text{viral_score}_w}{\text{avg_popularity}_{\text{genre}}} \quad (4)$$

Numerator aggregates Reddit engagement across all 5-minute windows in 48h. Denominator normalizes by genre-specific TMDb popularity baseline. Result > 1 means Reddit buzz exceeds expected TMDb levels.

Weighted engagement score:

$$\text{viral_score} = n_{\text{upvotes}} + (2 \times n_{\text{comments}}) + (10 \times n_{\text{awards}}) \quad (5)$$

Awards weighted $10\times$ (high user investment), comments $2\times$ (active discussion), upvotes $1\times$ (passive approval). This captures engagement depth, not just volume.

Viral status classification:

$$\text{status} = \begin{cases} \text{viral} & \text{if } V \geq 0.10 \\ \text{trending} & \text{if } 0.05 \leq V < 0.10 \\ \text{growing} & \text{if } 0.01 \leq V < 0.05 \\ \text{stable} & \text{if } V < 0.01 \end{cases} \quad (6)$$

Thresholds based on production data: viral = top 5% (explosive growth), trending = top 20% (strong momentum), growing = top 50% (moderate buzz).

Marketing opportunity score:

$$O = V \times (1 + \text{urgency}) \quad (7)$$

$$\text{urgency} = \text{recency} \times \text{momentum} \quad (8)$$

$$\text{recency} = e^{-\text{age_hours}/24}, \quad \text{momentum} = \frac{\text{velocity}_{\text{now}}}{\text{velocity}_{24h_ago}} \quad (9)$$

Recency uses exponential decay (24h half-life) to prioritize fresh discussions. Momentum compares current vs. 24h-ago velocity to detect acceleration. Combined urgency amplifies viral coefficient for time-sensitive opportunities.

Recommended action classification:

$$\text{action} = \begin{cases} \text{amplify immediately} & \text{if } O \geq 0.10 \text{ and urgency} \geq 0.95 \\ \text{monitor closely} & \text{if } O \geq 0.05 \\ \text{organic growth} & \text{if } O \geq 0.01 \\ \text{evaluate} & \text{if } O < 0.01 \end{cases} \quad (10)$$

Amplify immediately requires both high opportunity ($O \geq 0.10$) and high urgency (≥ 0.95) indicating fresh, accelerating buzz. Lower tiers prioritize by opportunity score magnitude, balancing intervention cost against potential reach.

Goal #3: Recommendations.

Intuition: Balance fleeting Reddit hype (60%) with enduring TMDB quality (40%) to recommend movies that perform well on both dimensions—avoiding one-hit wonders and hidden gems alike.

Dual-success score:

$$D_{dual} = 0.6 \times R_{Reddit} + 0.4 \times T_{TMDB} \quad (11)$$

where both R_{Reddit} and T_{TMDB} are normalized to 0-100 scale via min-max transformation:

$$\text{normalized} = \frac{x - x_{min}}{x_{max} - x_{min}} \times 100 \quad (12)$$

This ensures comparable scales before weighting. 60/40 split favors recent social buzz over historical quality, reflecting modern discovery patterns.

Reddit buzz raw score:

$$R_{raw} = W_{engagement} \times \text{decay}_{temporal} \times \text{multiplier}_{volume} \quad (13)$$

$$W_{engagement} = n_{upvotes} + (2 \times n_{comments}) + (10 \times n_{awards}) \quad (14)$$

$$\text{decay}_{temporal} = e^{-\text{age_hours}/24} \quad (15)$$

$$\text{multiplier}_{volume} = 1 + \frac{\log_{10}(n_{posts} + 1)}{10} \quad (16)$$

Temporal decay (24h half-life) ensures fresh discussions dominate. Volume multiplier (logarithmic) rewards sustained conversation without over-weighting spam—10 posts only slightly better than 5 posts.

TMDB quality raw score:

$$T_{raw} = 0.5 \times \text{popularity} + 0.3 \times (\text{vote_avg} \times 10) + 0.2 \times \log_{10}(\text{votes} + 1) \quad (17)$$

Hybrid formula: 50% popularity (current mainstream appeal), 30% rating (critical consensus, scaled to 0-100), 20% credibility (vote volume with logarithmic scaling to prevent outlier bias). Balances mass appeal with quality signals.

Lambda Merge Strategy. Speed layer (48h) prioritized when available, otherwise falls back to batch layer. Sentiment uses averaged windows, viral detection aggregates engagement across windows, recommendations blend recency-weighted Reddit metrics with TMDB quality signals. Target latency: p95 < 100ms via MongoDB indexes and connection pooling.

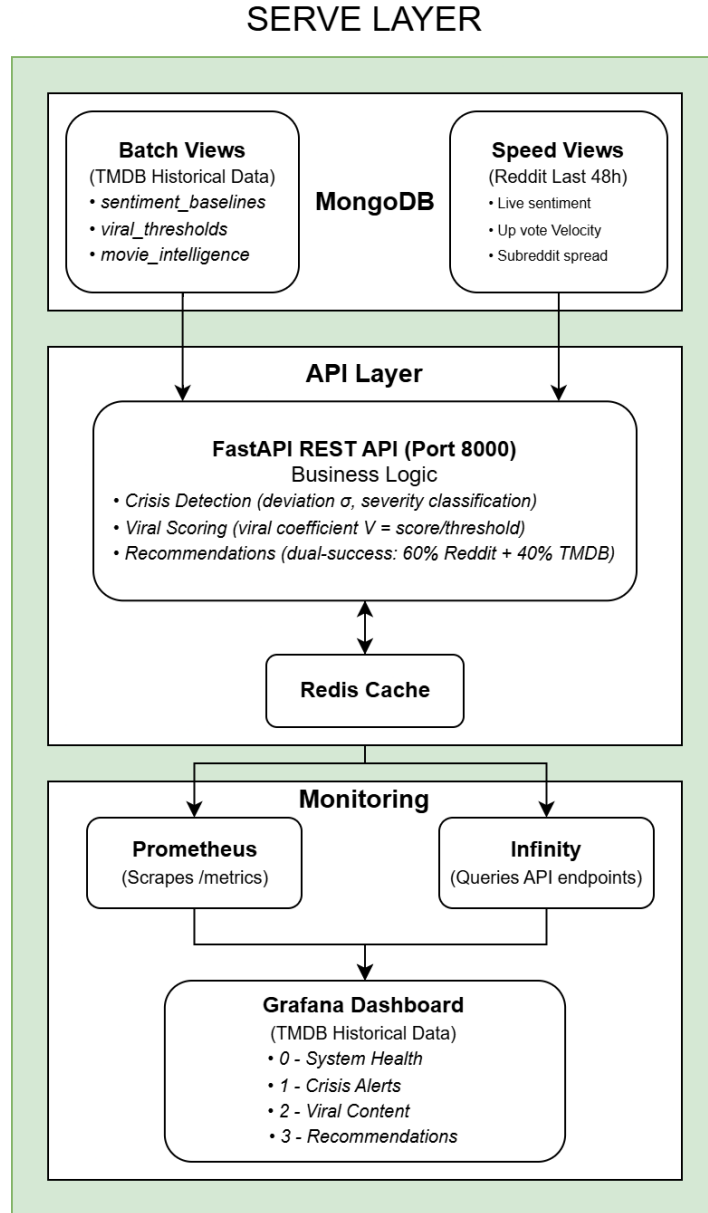


Figure 4: Serving Layer Data Flow and Interactions

3 Implementation Details

This section provides comprehensive technical documentation of the Movie Data Analysis Pipeline implementation. It details the source code structure, data flow mechanics, and core algorithmic logic, adhering to the Lambda Architecture paradigm to ensure robust processing of both historical and real-time data.

3.1 Source Code Documentation

The system architecture is divided into three distinct layers: Batch, Speed, and Serving. The following subsections delineate the directory structure, workflow orchestration, and implementation logic for each layer.

3.1.1 Batch Layer - Historical Data Processing

The Batch Layer is responsible for managing the master dataset and pre-computing historical views. It utilizes Apache Airflow for orchestration and Apache Spark for distributed processing.

Directory Structure

The codebase is organized to separate orchestration logic from data processing jobs:

```
layers/batch_layer/
├── airflow_dags/
│   └── tmdb_batch_pipeline.py ..... DAG orchestration logic
├── spark_jobs/
│   ├── bronze_ingest.py ..... TMDB API ingestion
│   ├── silver_transform.py ..... Baseline calculation
│   ├── gold_aggregate.py ..... Metadata enrichment
│   ├── export_to_mongo.py ..... MongoDB export
│   └── utils/
│       ├── spark_session.py
│       ├── s3_utils.py
│       └── logger.py
└── Dockerfile.airflow
```

Data Flow Architecture The batch processing workflow proceeds sequentially as follows:

1. **Orchestration:** The `tmdb_batch_pipeline.py` Airflow DAG triggers the pipeline execution.
2. **Ingestion (Bronze Layer):** The `bronze_ingest.py` script fetches movie and genre data from the TMDB API, adhering to a rate limit of 4 requests/second. Raw JSON data is persisted to MinIO as `tmdb_movies.parquet` and `tmdb_genres.parquet`.
3. **Transformation (Silver Layer):** The `silver_transform.py` job reads Bronze data to compute three key datasets:
 - *Sentiment Baselines:* Aggregations by genre, franchise, and year.
 - *Viral Thresholds:* P99 cutoffs categorized by budget tiers.
 - *Movie Intelligence:* Granular per-movie metrics.

These are written to MinIO as Silver Layer Parquet files.

4. **Enrichment (Gold Layer):** The `gold_aggregate.py` job appends temporal metadata (e.g., `batch_run_timestamp`) and persists the refined data.
5. **Export:** Finally, `export_to_mongo.py` performs a bulk upsert of the Gold datasets into three MongoDB collections: `sentiment_baselines`, `viral_thresholds`, and `movie_intelligence`.

Core Logic Implementation The batch layer processes TMDB data through a four-stage pipeline, with each stage implemented as a separate Python module. The following describes how these files interact and transform data:

1. Orchestration Layer: `tmdb_batch_pipeline.py`

This Airflow DAG file orchestrates the entire batch pipeline by defining task dependencies and execution order. It schedules the pipeline to run daily at 2 AM UTC with a maximum of one concurrent run.

Key Components:

- **DAG Configuration:** Defines retry strategy (2 retries with 5-minute delay), SLA (< 2 hours), and prevents catchup runs.
- **Task Executor:** The `run_spark_job()` function executes Spark jobs as subprocess calls:
 - Sets working directory to `/opt/airflow/spark_jobs`.
 - Passes `PYTHONPATH` environment variable for module imports.
 - Captures stdout/stderr for comprehensive logging.
 - Raises exceptions on non-zero exit codes with detailed error messages.
- **Complete Task Flow:** The pipeline includes logging and validation tasks:
 1. `metadata_start_log` → `fetch_tmdb_metadata` → `metadata_end_log`
 2. `baseline_start_log` → `calculate_baselines` → `baseline_end_log`
 3. `export_start_log` → `export_baselines` → `export_end_log`
 4. `mongo_start_log` → `export_to_mongo` → `mongo_end_log`
 5. `validate_pipeline` → `send_success_notification`

Each logging task records execution timestamps and task context for monitoring and debugging.

- **Environment Dependencies:** Requires environment variables:
 - `TMDB_API_KEY`: TMDB API authentication token.
 - `MINIO_BUCKET_BRONZE`, `MINIO_BUCKET_SILVER`, `MINIO_BUCKET_GOLD`: S3 bucket names.
 - `MONGODB_CONNECTION_STRING`: MongoDB connection URI for export stage.

2. Ingestion Layer: `bronze_ingest.py`

This module fetches raw data from TMDB API and persists it to MinIO storage in Parquet format. It implements the `TMDBAPIClient` class for API interaction and `BronzeIngestionJob` class for orchestration.

Data Flow:

- **Input:** TMDB API endpoints (`/genre/movie/list`, `/movie/{category}`).
- **Rate Limiting:** Enforces 4 requests/second using token bucket algorithm via `_rate_limit()` method.
- **Retry Strategy:** HTTP session configured with exponential backoff (3 retries, factor=2) for status codes 429, 500-504.
- **Data Extraction:** `fetch_genres()` retrieves genre mappings; `fetch_movies()` fetches movies from categories (popular, top_rated, now_playing, upcoming) up to 500 pages (10,000 movies per category).
- **Movie Enrichment Process:** Method `_enrich_movies_with_details()` performs detailed metadata fetching:
 1. For each movie, calls `/movie/{id}` endpoint to fetch budget, runtime, belongs_to_collection.

2. Calls `/movie/{id}/credits` endpoint to extract director from crew (filters `job="Director"`).
 3. Implements strict rate limiting: 40 requests per 10 seconds (2 API calls per movie).
 4. Progress logging every 100 movies with percentage completion.
 5. Fallback handling: If enrichment fails, retains movie with default values (`budget=0`, `runtime=None`, `director=None`).
 6. Estimated time: ~30 minutes for 1000 movies due to API rate constraints.
- **Review Fetching:** Method `_fetch_reviews_batch()` fetches reviews for sentiment baseline:
 - *Critical constraint:* Fetches reviews for top 50 movies after deduplication (not all movies).
 - Uses `/movie/{id}/reviews` endpoint with 1 page per movie.
 - Adds `movie_id` to each review for join operations in Silver layer.
 - This sampling approach maintains reasonable API usage while providing sentiment data for baseline calculation.
 - **Output:** Writes three Parquet files to MinIO Bronze layer:
 - `s3a://bronze-data/tmdb_movies/`: Enriched movie metadata with fields: `id`, `title`, `genre_ids`, `release_date`, `vote_average`, `vote_count`, `popularity`, `budget`, `runtime`, `belongs_to_collection`, `director`, `extraction_timestamp`.
 - `s3a://bronze-data/tmdb_genres/`: Genre mappings with fields: `id`, `name`, `extraction_timestamp`.
 - `s3a://bronze-data/tmdb_reviews/`: Reviews for top 50 movies with fields: `id`, `author`, `content`, `created_at`, `movie_id`, `extraction_timestamp`.

3. Transformation Layer: `silver_transform.py`

This module enriches Bronze data and generates three separate analytical datasets using the `BaselineCalculationJob` class.

Data Flow:

- **Input:** Reads Parquet files from MinIO Bronze layer (`tmdb_movies`, `tmdb_genres`, optionally `tmdb_reviews`).
- **Note:** Movies where Bronze layer enrichment failed have `budget=0`, resulting in "unknown" budget tier classification.
- **Enrichment Process:** Method `_enrich_movies()` joins movies with genres and derives:
 - `primary_genre`: First genre name from `genre_ids` array via join with `genres_df`.
 - `franchise`: Extracted from `belongs_to_collection` field (string, not object).
 - `budget_tier`: Classified as "indie" (<\$20M), "mid" (\$20M-\$100M), or "blockbuster" (>\$100M), or "unknown" if `budget=0`.
 - `release_year`, `release_month`, `season`: Temporal features from `release_date`.
 - * Season mapping: Winter (Dec, Jan, Feb - crosses year boundary), Spring (Mar, Apr, May), Summer (Jun, Jul, Aug), Fall (Sep, Oct, Nov).
 - `director`: Fetched during Bronze layer enrichment via credits API; Silver layer includes defen-

sive null check.

– **Sentiment Calculation:** Two-stage approach:

1. If reviews available: Applies `sentiment_score` UDF to review content, aggregates `avg_sentiment` and `review_count` per movie.
2. If reviews unavailable: Uses rating as proxy: $\text{avg_sentiment} = (\text{vote_average} - 5) / 5$ (normalizes 0-10 scale to -1 to 1 range).

This fallback mechanism ensures all movies have sentiment scores even without review data.

• **Dataset Generation:** Three separate methods produce distinct outputs:

1. `_generate_sentiment_baselines()`: Applies critical filter to ensure baseline quality - only includes movies with actual review data (`review_count > 0` AND `avg_sentiment ≠ 0`), preventing movies with rating-proxy sentiments from diluting baselines. Then aggregates by three dimensions:

- Genre-level: `avg_sentiment`, `sentiment_stddev`, `movie_count`, `review_count` grouped by `primary_genre`.
- Franchise-level: Same metrics grouped by `franchise` (nulls out genre field).
- Year-level: Same metrics grouped by `release_year` (nulls out genre/franchise fields).

Result: Single union of all baselines with mutually exclusive dimension fields.

2. `_generate_viral_thresholds()`: Calculates 99th percentile thresholds by three dimensions:

- Genre-level: P99 `vote_count`, `avg_popularity`, `movie_count` grouped by `primary_genre`.
- Budget-tier-level: P99 `vote_count` grouped by `budget_tier` (excludes "unknown").
- Season-level: P99 `vote_count` grouped by `season`.

Result: Union of thresholds with mutually exclusive dimension fields.

3. `_generate_movie_intelligence()`: Selects per-movie competitive data:

- Fields: `movie_id`, `title`, `genre` (as array), `release_date`, `release_year`, `avg_sentiment`, `vote_average`, `vote_count`, `popularity`, `budget`, `budget_tier`, `franchise`, `director`, `runtime`, `review_count`.

• **Output:** Writes three separate Parquet datasets to MinIO Silver layer:

- `s3a://silver-data/sentiment_baselines/`
- `s3a://silver-data/viral_thresholds/`
- `s3a://silver-data/movie_intelligence/`

4. Aggregation Layer: `gold_aggregate.py`

This module adds temporal metadata to Silver datasets without merging them, implemented via the `GoldAggregationJob` class.

Data Flow:

- **Input:** Reads three separate Parquet datasets from MinIO Silver layer.

- **Metadata Enrichment:** Method `_add_metadata()` appends:
 - `batch_run_timestamp`: ISO 8601 UTC timestamp of processing run.
 - `aggregation_granularity`: "all_time" (enables future daily/weekly aggregations).
 - `data_period_start`, `data_period_end`: Date range of aggregated data.
- **Schema Normalization:** Converts genre arrays to comma-separated strings for consistency.
- **Output:** Writes three separate Parquet datasets to MinIO Gold layer (no merging):
 - `s3a://gold-data/sentiment_baselines/`
 - `s3a://gold-data/viral_thresholds/`
 - `s3a://gold-data/movie_intelligence/`

5. Export Layer: `export_to_mongo.py`

This module exports Gold data to MongoDB collections for serving layer access, using the `MongoDBExport` class.

Data Flow:

- **Input:** Reads three separate Parquet datasets from MinIO Gold layer using Spark.
- **Index Creation:** Method `create_indexes()` builds collection-specific indexes:
 - `sentiment_baselines`: Indexes on genre, franchise, year, (genre+year), (franchise+year), `batch_run_timestamp`.
 - `viral_thresholds`: Indexes on genre, budget_tier, season, (genre+budget_tier+season), `batch_run_timestamp`.
 - `movie_intelligence`: Unique index on `movie_id`, compound indexes on (genre+release_year), `budget_tier`, `franchise`, `batch_run_timestamp`.
- **Bulk Upsert:** Method `export_collection()` performs bulk upsert operations:
 - Converts Spark Row objects to Python dictionaries via `_row_to_dict()`.
 - Uses collection-specific filter keys for upsert matching (`movie_id` for `movie_intelligence`, dimension fields for `baselines/thresholds`).
 - Batch size: 1000 documents per bulk operation.
 - Error handling: Continues on `BulkWriteError`, logs failed documents.
- **Output:** Updates three MongoDB collections in `moviedb` database:
 - `sentiment_baselines`: One document per genre/franchise/year dimension.
 - `viral_thresholds`: One document per genre/budget_tier/season dimension.
 - `movie_intelligence`: One document per movie (3,260+ documents).

Inter-File Data Dependencies

- All Spark jobs use shared utilities from `utils/` directory:
 - `spark_session.py`: Creates `SparkSession` with MinIO S3A configuration.
 - * Configures S3A filesystem with MinIO endpoints.
 - * Sets Spark memory and executor settings.

- * Enables Hive support for Parquet metadata.
- `s3_utils.py`: Generates S3A paths for Bronze/Silver/Gold layers.
 - * Path builder pattern: `S3PathBuilder(bucket, prefix).with_partition().build()`.
 - * Functions: `get_bronze_path()`, `get_silver_path()`, `get_gold_path()`.
 - * Supports date partitioning: `year=2025/month=12/day=28/hour=14/`.
 - * Bucket resolution from environment variables or defaults.
- `logger.py`: Provides structured logging with execution metrics.
 - * `JobMetrics` class tracks: records processed, execution time, errors.
 - * Structured logging with context: `logger.info(msg, extra={"context": {...}})`.
 - * Decorator `@log_execution` for automatic job timing and error capture.
- **Data Lineage and File Interactions:**
 1. `bronze_ingest.py` writes Parquet files to MinIO Bronze bucket:
 - File: `s3a://bronze-data/tmdb_movies/*.parquet` (partitioned by extraction date).
 - File: `s3a://bronze-data/tmdb_genres/*.parquet`.
 - File: `s3a://bronze-data/tmdb_reviews/*.parquet`.
 2. `silver_transform.py` reads Bronze Parquet files, performs transformations, writes to Silver:
 - Reads: Bronze movies, genres, reviews using `spark.read.parquet()`.
 - Joins: Movies $\bowtie$ Genres on `genre_id`, Movies $\bowtie$ Reviews on `movie_id`.
 - Aggregates: GroupBy operations for baselines and thresholds.
 - Writes: Three separate Parquet datasets to Silver bucket (no co-location).
 3. `gold_aggregate.py` reads Silver datasets, adds metadata, writes to Gold:
 - No data transformation - only metadata enrichment.
 - Preserves three-dataset separation (`sentiment_baselines`, `viral_thresholds`, `movie_intelligence`).
 - Converts genre arrays to comma-separated strings for MongoDB compatibility.
 4. `export_to_mongo.py` reads Gold Parquet, converts to MongoDB documents:
 - Uses `spark.read.parquet()` to load Gold datasets.
 - Converts Spark Row objects to Python dicts via recursive `_row_to_dict()`.
 - Creates collection-specific indexes before bulk upsert.
 - Uses `ReplaceOne(filter, doc, upsert=True)` for idempotent writes.
- Each stage reads from previous stage's MinIO bucket and writes to next stage's bucket, ensuring data lineage, reproducibility, and fault tolerance through immutable storage.

Batch Layer Data Point Structure The batch layer produces three types of MongoDB collections, each with specific data schemas:

1. Movie Intelligence Collection

Comprehensive movie analytics with 3,260+ documents.

Key Features:

- `movie_id`: TMDB identifier.
- `title`, `genre`, `release_year`: Movie metadata.
- `avg_sentiment`: Historical sentiment score (-1.0 to 1.0).
- `vote_average`, `vote_count`: TMDB ratings.
- `popularity`: TMDB popularity metric.
- `budget`, `budget_tier`: Budget classification (indie/mid/blockbuster).
- `franchise`, `director`, `runtime`: Extended metadata.
- `batch_run_timestamp`: Temporal metadata for versioning.

2. Sentiment Baselines Collection

Pre-calculated statistical baselines for deviation analysis.

Key Features:

- Dimension fields (mutually exclusive per row):
 - `genre`: Genre-level baseline (e.g., "Action").
 - `franchise`: Franchise-level baseline (e.g., "Marvel Cinematic Universe").
 - `year`: Year-level baseline (e.g., 2023).
- `avg_sentiment`: Mean sentiment for segment.
- `sentiment_stddev`: Standard deviation (for σ calculation).
- `movie_count`, `review_count`: Sample size indicators.
- Each row contains only **one dimension** (others are null) enabling flexible baseline comparison strategies.

3. Viral Thresholds Collection

Context-aware thresholds for viral detection.

Key Features:

- Dimension fields (mutually exclusive per row):
 - `genre`: Genre-specific threshold.
 - `budget_tier`: Budget tier-specific threshold.
 - `season`: Seasonal threshold.
- `avg_popularity`: Average TMDB popularity (threshold baseline).
- `viral_threshold`: P99 vote count threshold.
- `movie_count`: Sample size for threshold calculation.
- Single-dimension rows allow context-specific viral detection without multi-dimensional explo-

sion.

3.1.2 Speed Layer - Real-Time Data Processing

Directory Structure

```
layers/speed_layer/
├── reddit_producers/
│   ├── reddit_stream_producer.py ..... Reddit JSON scraper
│   ├── movie_matcher.py ..... TMDB title validator
│   └── schemas.py ..... Data schemas
├── streaming_jobs/
│   └── reddit_sentiment_stream.py ..... Spark Streaming job
├── connectors/
│   └── reddit_sync.py ..... Cassandra→MongoDB sync
├── cassandra_views/
│   └── reddit_schema.cql ..... Cassandra DDL
└── kafka_producers/
    └── setup_topics.py ..... Kafka topic setup
```

Data Flow Architecture The real-time processing pipeline operates as follows:

1. **Ingestion:** The `reddit_stream_producer.py` scrapes Reddit JSON endpoints using the `.json` trick (appending `?limit=100` to subreddit URLs) without requiring API authentication (subreddits: `r/movies`, `r/boxoffice`, `r/TrueFilm`) every 30 seconds.
2. **Validation:** The `movie_matcher.py` module validates extracted titles against a TMDB cache before publishing JSON messages to Kafka topics (`reddit.posts`, `reddit.comments`).
3. **Processing:** The `reddit_sentiment_stream.py` job consumes Kafka streams, applies VADER sentiment analysis via UDF, and aggregates metrics in 5-minute tumbling windows.
4. **Storage:** Results are written to two Cassandra tables (`reddit_post_metrics` and `reddit_comment_metrics`) with a 48-hour Time-To-Live (TTL).
5. **Synchronization:** The `reddit_sync.py` connector reads from both Cassandra tables and merges them into the MongoDB `speed_views` collection every 5 minutes, using the `data_type` field (`"reddit_post"` or `"reddit_comment"`) to distinguish the source.

Core Logic Implementation The speed layer processes Reddit data through a multi-component real-time pipeline. The following describes how these files interact and stream data:

1. Data Ingestion: `reddit_stream_producer.py`

This producer uses the `.json` scraping technique and publishes validated movie mentions to Kafka. It implements the `RedditStreamProducer` class with infinite loop execution.

Continuous Operation Mode:

Data Fetching Method:

- Appends `.json` to Reddit URLs: `https://www.reddit.com/r/{subreddit}/new.json?limit=100`

- Returns JSON payloads without needing PRAW or official API credentials
- Requests library used instead of `requests-oauthlib`
- Custom User-Agent header required to avoid blocking

Rate Limiting Implementation:

- Enforces minimum 2-second delay between requests (line 132-136)
- Prevents aggressive scraping and IP blocking
- Tracks `request_count` and `last_request_time`

Deduplication Strategy:

- Maintains in-memory sets: `seen_posts` and `seen_comments`
- Skips posts/comments already processed (prevents duplicate Kafka messages)
- Note: Sets are cleared on restart; consider persisting to Redis for stateless deployments

Data Flow:

- **Input Source:** Reddit JSON endpoints (no authentication required):
 - `https://reddit.com/r/{subreddit}/new/.json`: New posts from `r/movies`, `r/boxoffice`, `r/TrueFilm`.
 - `https://reddit.com/r/{subreddit}/comments/.json`: New comments from same subreddits.
- **Polling Strategy:** 30-second intervals with 2-second rate limiting between requests.
- **Movie Extraction:** Method `_extract_movie_titles()` uses regex patterns:
 - Quoted strings: `r' " ([^"]+)" '`.
 - Title case sequences: `r'\b ([A-Z] [a-z]+(?:\s+[A-Z] [a-z]+)*) \b'`.
 - Multi-word titles (2-6 words): `r'\b ([A-Z] [a-z]+(?:\s+[A-Z] [a-zA-Z]+){1,5}) \b'`.

Extracts up to 20 candidates per post/comment.

- **TMDB Validation:** Integrates `MovieMatcher` class with sophisticated caching:
 - **Cache Management:**
 - * Loads TMDB movie cache (10,000+ titles) on initialization.
 - * Automatic cache refresh every 24 hours (TTL-based).
 - * Stores as dictionary: `{normalized_title: (movie_id, original_title, year)}`.
 - * Cache source: TMDB API or batch layer MongoDB collection.
 - **Title Normalization Process:**
 1. Extracts and removes year from title (e.g., "Dune (2021)" → "dune").
 2. Converts to lowercase.
 3. Removes special characters except spaces and numbers (preserves sequel numbers like "2", "3").
 4. Removes leading articles ("the", "a", "an").
 5. Collapses multiple spaces to single space.

- **Fuzzy Matching Algorithm:**
 - * Uses Python's `difflib.SequenceMatcher` for similarity calculation.
 - * Compares normalized extracted title against all cached titles.
 - * Returns best match if similarity ≥ 0.8 threshold.
 - * Handles title variations (e.g., "The Matrix" $\leftrightarrow$ "Matrix").
- Returns canonical TMDb title for validated matches with confidence score.
- Logs validation results for debugging (e.g., "Dune: Part Two" $\rightarrow$ "Dune: Part Two" score: 1.0).
- **Deduplication:** Maintains in-memory sets `seen_posts` and `seen_comments` to prevent duplicate processing.
- **Output:** Publishes JSON messages to Kafka topics:
 - `reddit.posts`: Post data with fields: `post_id`, `subreddit`, `title`, `selftext`, `author`, `created_utc`, `upvotes`, `upvote_ratio`, `num_comments`, `awards`, `url`, `is_self`, `potential_movies` (validated TMDb titles), `fetch_at`.
 - `reddit.comments`: Comment data with fields: `comment_id`, `post_id`, `subreddit`, `body`, `author`, `created_utc`, `upvotes`, `awards`, `parent_id`, `is_submitter`, `potential_movies`, `fetch_at`.
- **Message Format:** Gzip-compressed JSON with `acks='all'` for delivery guarantee.

2. Stream Processing: `reddit_sentiment_stream.py`

This Spark Structured Streaming job consumes Kafka streams, performs sentiment analysis, and writes aggregated metrics to Cassandra. Implemented via `RedditSentimentStreaming` class with continuous execution mode.

Configuration and Dependencies:

- **Checkpoint Location:**
 - General checkpoint location: `/app/checkpoints/reddit_stream`.
 - Per-table checkpoints: `/app/checkpoints/reddit_post_metrics` and `/app/checkpoints/reddit_comment_metrics`.
 - Stores Kafka offsets (exactly-once source semantics).
 - Stores watermark state (for late event handling).
 - Stores aggregation state (for stateful operations).
 - Enables fault tolerance: restarts from last committed offset on failure.
- **Execution Mode:**
 - Continuous micro-batch processing (default trigger interval).
 - Runs indefinitely until manual termination.
 - Graceful shutdown on `SIGTERM`/`SIGINT` signals.

Data Flow:

- **Input:** Reads from two Kafka topics (`reddit.posts`, `reddit.comments`) using Spark Kafka connector.

- **Schema Parsing:** Methods `read_reddit_posts_stream()` and `read_reddit_comments_stream()` parse JSON using predefined schemas:

- REDDIT_POST_SCHEMA: 14 fields including `potential_movies` (ArrayType).
- REDDIT_COMMENT_SCHEMA: 12 fields including `potential_movies` (ArrayType).

Converts Unix timestamp `created_utc` to `event_time` for watermarking.

- **Sentiment Analysis:** Creates UDFs via `create_post_sentiment_udf()` and `create_comment_sentiment_udf()`:

- Uses VADER SentimentIntensityAnalyzer (vaderSentiment library).
- For posts: Analyzes concatenated title + selftext.
- For comments: Analyzes comment body.
- Returns compound score (-1.0 to 1.0): negative, neutral, or positive.
- Handles None/empty text gracefully (returns 0.0 neutral).

- **Window Aggregation:** Method `process_posts_with_sentiment()` implements:

- Explodes `potential_movies` array to one row per movie mention.
- Applies 5-minute tumbling windows with 30-second watermark.
- Groups by (window, movie_title).
- Aggregates: `post_count`, `total_upvotes`, `avg_upvote_ratio`, `total_comments`, `total_awards`, `avg_sentiment`, `max_upvotes`.
- Calculates viral metrics:
 - * `upvote_velocity = total_upvotes / 300 seconds`.
 - * `comment_velocity = total_comments / 300 seconds`.
 - * `award_velocity = total_awards / 300 seconds`.
 - * `viral_score = (upvote_velocity × 0.5) + (comment_velocity × 0.3) + (award_velocity × 0.2)`.
- Adds metadata: `window_start`, `hour` (truncated to hour), `data_source="reddit"`, `processed_at`.

Similar process for comments via `process_comments_with_sentiment()` with metrics: `comment_count`, `total_upvotes`, `total_awards`, `avg_sentiment`, `max_upvotes`.

- **Output:** Writes streaming results to Cassandra using Spark Cassandra Connector:

- **Schema Design:**

- * Tables: `speed_layer.reddit_post_metrics` and `speed_layer.reddit_comment_metrics`.
- * Composite partition key: (movie_title, hour) for temporal distribution.
 - Distributes writes across cluster nodes based on movie and hour.
 - Prevents hot partitions during viral events (spreads load).
 - Example: "Dune_2024-01-15T14" and "Dune_2024-01-15T15" stored on different nodes.
- * Clustering key: `window_start DESC` for chronological ordering within partition.
 - Enables efficient time-range queries (e.g., last 24 hours for a movie).

- Newer windows appear first in query results.
- * TTL: 48 hours configured in schema to auto-purge stale data.
 - Automatically removes old data after 2 days.
 - Prevents unbounded storage growth.
 - Set via `default_time_to_live = 172800` in `reddit_schema.cql`.
- **Write Pattern:**
 - * Output mode: "append" (new windows only, no updates to existing data).
 - * `foreach` sink for exactly-once semantics.
 - * Coordinated with Spark checkpointing to prevent duplicates on failure recovery.
 - * Each micro-batch writes complete 5-minute window results atomically.
- **Checkpointing:**
 - * Checkpoint locations: `/app/checkpoints/reddit_post_metrics` and `/app/checkpoints/reddit_comment_metrics`.
 - * Stores Kafka offsets, watermark state, and aggregation state.
 - * Enables exactly-once processing guarantee on failure recovery.
- Partitioning strategy handles high-velocity movies (e.g., blockbuster releases with 100+ posts/hour).
- Schema defined in `reddit_schema.cql` with appropriate data types (text, timestamp, int, double).

3. Data Synchronization: `reddit_sync.py`

This connector periodically syncs Cassandra speed views to MongoDB for serving layer access, using the `RedditCassandraToMongoSync` class.

Data Flow:

- **Input:** Reads from two Cassandra tables:
 - `speed_layer.reddit_post_metrics`: Query selects all columns (`movie_title`, `hour`, `window_start`, `post_count`, `total_upvotes`, `avg_upvote_ratio`, `total_comments`, `total_awards`, `avg_sentiment`, `max_upvotes`, `upvote_velocity`, `comment_velocity`, `award_velocity`, `viral_score`, `data_source`, `processed_at`).
 - `speed_layer.reddit_comment_metrics`: Query selects columns (`movie_title`, `hour`, `window_start`, `comment_count`, `total_upvotes`, `total_awards`, `avg_sentiment`, `max_upvotes`, `data_source`, `processed_at`).
- **Data Transformation:** Methods `_sync_post_metrics()` and `_sync_comment_metrics()`:
 - Convert Cassandra Row objects to Python dictionaries.
 - Nest metric fields under `metrics` key for cleaner API structure.
 - Add `data_type` field: "reddit_post" or "reddit_comment" to distinguish sources.
 - Add `synced_at`: Current sync timestamp.
 - Add `ttd_expires_at`: `window_start` + 48 hours for automatic expiration.

- **MongoDB Upsert:** Uses bulk operations:
 - Upsert filter: `{movie_title, data_type, hour}` uniquely identifies documents.
 - Batch size: 1000 documents per bulk operation.
 - Error handling: Logs `BulkWriteError` details, continues processing.
- **Index Management:** Creates MongoDB indexes for efficient queries.
- **Output:** Updates MongoDB collection `moviedb.speed_views`:
 - One document per `(movie_title, data_type, hour)` combination.
 - Active documents: 461+ (varies based on 48-hour TTL window).
 - Automatic cleanup: MongoDB TTL monitor deletes expired documents.
- **Sync Schedule:** Runs as continuous daemon with 5-minute intervals (configurable).

4. Topic Management: `setup_topics.py`

This utility script initializes Kafka topics with appropriate configurations for the speed layer.

Setup Process:

- Creates topics: `reddit.posts`, `reddit.comments`.
- Configuration: 3 partitions, replication factor 1, retention 48 hours.
- Ensures topics exist before producer starts publishing.

Inter-File Data Dependencies

- `reddit_stream_producer.py` depends on `movie_matcher.py` for TMDb validation:
 - `MovieMatcher` loads movie cache from batch layer MongoDB collection.
 - Uses fuzzy string matching (`RapidFuzz` library) to validate extracted titles.
 - Maintains cache in memory for fast lookups during high-frequency polling.
- Data flows through three storage systems:
 - Reddit JSON endpoint → Kafka topics (`reddit.posts`, `reddit.comments`).
 - Kafka → Spark Streaming → Cassandra tables (`reddit_post_metrics`, `reddit_comment_metrics`).
 - Cassandra → Sync connector → MongoDB collection (`speed_views`).
- Storage rationale:
 - Kafka: Decouples ingestion from processing, enables replay for recovery.
 - Cassandra: Optimized for time-series writes with automatic TTL expiration.
 - MongoDB: Provides flexible querying for serving layer API with nested documents.
- All components share common schemas defined in `schemas.py`:
 - `RedditPostSchema`: Dataclass for post validation.
 - `RedditCommentSchema`: Dataclass for comment validation.
 - Ensures consistency across producer, streaming job, and sync connector.

Speed Layer Data Point Structure The speed layer maintains real-time aggregated metrics in the `speed_views` collection with automatic 48-hour TTL expiration.

Collection: `speed_views` (461+ active documents)

MongoDB Indexes:

- Compound index: (`movie_title`, `data_type`, `hour`) for time-series lookups by movie.
- Index: (`data_type`, `hour`) for global time-series queries across all movies.
- TTL index: (`ttl_expires_at`) with `expireAfterSeconds=0` for automatic document expiration.
- Index: (`data_type`, `metrics.viral_score` DESC) for trending/viral content queries.
- Index: (`data_type`, `metrics.avg_sentiment` DESC) for sentiment-based ranking.

Key Features:

- `window_start`, `hour`: Temporal partitioning (5-minute windows aggregated by hour).
- `movie_title`: Movie identifier for matching with batch layer.
- `data_type`: "reddit_post" or "reddit_comment".
- `data_source`: "reddit" (extensible for other platforms).
- `metrics`: Nested object with `data_type`-dependent fields:
 - For `reddit_post`:
 - * `post_count`: Number of posts in window (1-2 typical).
 - * `total_upvotes`: Sum of upvotes (-13 to 2812 range, can be negative).
 - * `avg_upvote_ratio`: Average upvote ratio (0.0-1.0).
 - * `total_comments`: Total comment count on posts.
 - * `total_awards`: Reddit awards received.
 - * `avg_sentiment`: VADER sentiment score (-1.0 to 1.0).
 - * `max_upvotes`: Peak upvotes in single post.
 - * `upvote_velocity`, `comment_velocity`, `award_velocity`: Engagement rates.
 - * `viral_score`: Composite virality metric.
 - For `reddit_comment`:
 - * `comment_count`: Number of comments in window.
 - * `total_upvotes`: Sum of comment upvotes.
 - * `total_awards`: Reddit awards on comments.
 - * `avg_sentiment`: VADER sentiment score.
 - * `max_upvotes`: Peak comment upvotes.
- `processed_at`: Streaming job processing timestamp.
- `synced_at`: MongoDB synchronization timestamp.
- `ttl_expires_at`: Auto-deletion timestamp (48h from creation).

Storage Strategy:

- Primary storage: Cassandra two tables:
 - reddit_post_metrics: Post-specific viral metrics
 - reddit_comment_metrics: Comment-specific engagement metrics
- Secondary storage: MongoDB speed_views collection (synchronized from Cassandra tables for serving layer queries).
- TTL: 48 hours automatic expiration (no manual cleanup required).
- Partitioning: By movie_title and hour for efficient time-range queries.
- Data structure: Flat fields in Cassandra, nested metrics object in MongoDB for cleaner API responses.

3.1.3 Serving Layer - Query-Time Merge and Analytics

Directory Structure

The directory structure has been expanded to include monitoring configurations and visualization assets:

```
layers/serving_layer/
├── api/
│   ├── main.py ..... FastAPI application
│   └── routes/
│       ├── health.py ..... Health check endpoints
│       ├── crisis_detection.py ..... PR crisis endpoints
│       ├── viral_detection.py ..... Viral content endpoints
│       ├── recommendations.py ..... Recommendation endpoints
│       └── utilities.py ..... Utility endpoints
│   └── schemas/
│       ├── crisis_detection.py ..... Pydantic models
│       ├── recommendations.py ..... Recommendation response models
│       ├── viral_detection.py ..... Viral detection response models
│       └── utilities.py ..... Utility response models
├── mongodb/
│   ├── client.py ..... MongoDB connection
│   ├── indexes.py ..... Index definitions
│   └── queries.py ..... Query abstractions
├── query_engine/
│   ├── similarity_engine.py ..... Similarity calculations
│   └── utils.py ..... Text processing utilities
├── monitoring/
│   ├── prometheus.yml ..... Metrics collection config
│   └── alerts.yml ..... Alert rules
├── visualization/
│   ├── grafana/
│   │   ├── dashboards/
│   │   │   ├── 0-system-health-overview.json
│   │   │   ├── 1-crisis-alert-overview.json
│   │   │   ├── 2-viral-content-detection.json
│   │   │   ├── 3-recommendation-optimization.json
│   │   │   └── dashboard-provider.yml
│   │   └── datasources/
│   │       └── datasource.yml ..... Prometheus datasource
│   └── provisioning/ ..... Grafana provisioning configs
└── Dockerfile
```


Data Flow Architecture The request lifecycle is managed as follows:

1. **Request Handling:** A client HTTP request is received by the FastAPI router (`api/main.py`) and directed to the appropriate endpoint based on URL path (e.g., `/api/v1/crisis-detection/*`, `/api/v1/viral-detection/*`, `/api/v1/recommendations/*`, or `/api/v1/health`).
2. **Query Routing:** The route handler in `api/routes/*.py` analyzes the request type and determines the optimal data source strategy by directly executing MongoDB queries:
 - *Real-time queries:* Speed layer only (`speed_views` collection).
 - *Historical analytics:* Batch layer only (`movie_intelligence`, `sentiment_baselines`, `viral_thresholds` collections).
 - *Hybrid queries:* Merges both layers inline for comprehensive analysis.
3. **Data Retrieval:** The `mongodb/queries.py` module executes optimized MongoDB queries using pre-built indexes (`mongodb/indexes.py`) to fetch relevant data from appropriate collections.
4. **Layer Merging & Analysis:** For hybrid queries, merging logic integrated into the route handlers normalizes titles using `query_engine/utils.py` fuzzy title matching. Route-specific logic then applies:
 - *Crisis Detection:* Calculates deviation sigma (σ) comparing current sentiment vs. baseline (using both franchise and genre baselines).
 - *Viral Detection:* Computes viral coefficient (Reddit engagement / TMDB popularity).
 - *Recommendations:* Calculates dual-success score (60% Reddit buzz + 40% TMDB quality).
5. **Response Validation:** Results are validated against Pydantic schemas (`api/schemas/*`) ensuring type safety and data integrity.
6. **Monitoring:** Prometheus scrapes metrics from the `/metrics` endpoint (if configured), tracking request latency, query performance, and error rates. Alert rules defined in `monitoring/alerts.yml`.

Core Logic Implementation

The serving layer implements a modular architecture with clear separation of concerns across multiple file components. This section describes the file-level structure and how data flows through these components to implement the three core business goals.

Architecture Components and File Interactions

1. Entry Point and Request Routing (`api/main.py`)

The `api/main.py` file serves as the application entry point, bootstrapping the FastAPI framework and registering five specialized route modules. During application startup, it initializes the Redis cache connection via the `query_engine/cache.py` module's `get_redis()` function. The file implements a lifespan context manager that handles graceful startup and shutdown, ensuring Redis connections are properly established at startup and closed at shutdown.

The application registers route modules with a `/api/v1` prefix:

- `api/routes/health.py`: System health monitoring endpoints.
- `api/routes/crisis_detection.py`: Goal #1 PR crisis detection endpoints.
- `api/routes/viral_detection.py`: Goal #2 viral content identification endpoints.
- `api/routes/recommendations.py`: Goal #3 recommendation optimization endpoints.
- `api/routes/utilities.py`: Utility and exploration endpoints.

Each incoming HTTP request is routed to the appropriate module based on the URL path, where specialized handlers implement the business logic.

2. Database Connection Management (`mongodb/client.py`)

The `mongodb/client.py` module manages the MongoDB connection lifecycle using a singleton pattern.

It exposes three primary functions:

- `get_mongodb_client()`: Returns a persistent `MongoClient` instance with connection pooling (100 max connections, 10 min connections).
- `get_database()`: Returns the specific database instance (default: `moviedb`).
- `check_mongodb_health()`: Performs health checks by pinging the server and measuring latency.

Route handlers invoke `get_mongodb_client()` to obtain database access, which is then used to query four primary collections: `sentiment_baselines`, `viral_thresholds`, `movie_intelligence` (batch layer), and `speed_views` (speed layer).

3. Query Abstraction Layer (`mongodb/queries.py`)

The `mongodb/queries.py` file defines the `MovieQueries` class, which encapsulates all database query patterns. This class receives a database instance in its constructor and provides methods organized by collection:

Batch Layer Query Methods:

- `get_sentiment_baselines()`: Queries `sentiment_baselines` collection with optional filters (genre, franchise, year).
- `get_viral_thresholds()`: Queries `viral_thresholds` collection with optional filters (genre, budget_tier, season).
- `get_batch_movie_by_id()`: Retrieves individual movie from `movie_intelligence` by TMDb ID.
- `get_movie_id_by_title()`: Performs fuzzy title matching on `movie_intelligence` using regex.

Speed Layer Query Methods:

- `get_speed_sentiment()`: Queries `speed_views` for recent sentiment data with configurable time windows.
- `get_speed_trending()`: Uses aggregation pipelines on `speed_views` to identify trending content.

Hybrid Query Methods:

- `get_movie_complete_view()`: Merges batch and speed layer data for a single movie.
- `get_batch_movies_for_recommendations()`: Retrieves candidate movies from batch layer with quality filters.
- `get_speed_layer_engagement()`: Aggregates speed layer engagement metrics by movie title for merging.

Route handlers instantiate `MovieQueries` and invoke these methods rather than writing raw MongoDB queries, providing abstraction and reusability.

4. Index Optimization (`mongodb/indexes.py`)

The `mongodb/indexes.py` file defines the `IndexManager` class, which creates and manages MongoDB indexes for optimal query performance. The `create_all_indexes()` method creates specialized indexes for each collection:

Sentiment Baselines Indexes:

- Single-field indexes: `genre`, `franchise`, `year`, `batch_run_timestamp`.
- Composite indexes: `(genre, year)`, `(franchise, year)` for combined filtering.

Viral Thresholds Indexes:

- Single-field indexes: `genre`, `budget_tier`, `season`, `batch_run_timestamp`.
- Composite index: `(genre, budget_tier, season)` for multi-dimensional queries.

Movie Intelligence Indexes:

- Primary key: `movie_id`.
- Search indexes: `title (text)`, `genre`, `release_year`.
- Composite index: `(genre, release_year, vote_average)` for recommendation filtering.

Speed Views Indexes:

- Temporal index: `window_start (descending)` for time-based queries.
- Composite index: `(movie_title, window_start)` for per-movie time series.

These indexes are created during deployment initialization and directly support the query patterns in `mongodb/queries.py`.

5. Title Normalization for Layer Merging

For cross-layer data merging, route modules (`crisis_detection.py`, `viral_detection.py`) each define a local `normalize_movie_title()` function that performs title normalization. This function:

- Removes special characters using regex pattern `[\w\s]`.
- Converts to lowercase.
- Normalizes whitespace (collapses multiple spaces to single space).

The normalization enables matching between batch layer titles (TMDB canonical titles) and speed layer titles (extracted from Reddit post text, often with variations). Note: The `query_engine/utils.py` module contains similar utility functions (`normalize_title()`, `fuzzy_match_title()`, `ex-`

`tract_base_title()`) that are available but currently not used by the main route handlers.

6. Response Caching Layer (`query_engine/cache.py`)

The `query_engine/cache.py` module implements a Redis-based caching decorator for API responses.

It provides:

- `get_redis()`: Returns async Redis client with connection pooling (50 max connections).
- `cached(ttl, prefix)`: Decorator that caches function return values with configurable TTL.
- `generate_cache_key()`: Creates deterministic keys from function name and arguments.

Route handlers in `recommendations.py` use the `@cached` decorator (e.g., `@cached(ttl=300, prefix="recommendations")`) to cache expensive queries. The decorator intercepts function calls, checks Redis for existing results, and falls back to function execution on cache miss. This reduces database load for frequently accessed endpoints like `/dual-success` and `/similar`.

7. Similarity Calculation Engine (`query_engine/similarity_engine.py`)

The `query_engine/similarity_engine.py` module implements content-based filtering for movie recommendations using cosine similarity. It provides:

- `build_feature_vector()`: Extracts 5 features from movie documents (genre, director, franchise, budget_tier, release_year).
- `calculate_feature_similarity()`: Computes weighted similarity scores with franchise dominant (53%), director important (27%), and other features contributing (20%).
- `calculate_similarity_score()`: Applies cosine similarity between feature vectors.

The `api/routes/recommendations.py` module imports these functions to implement the `/similar/{movie_id}` endpoint, which retrieves candidate movies from `movie_intelligence`, builds feature vectors for each, calculates pairwise similarities, and ranks results.

8. Pydantic Response Models (`api/schemas/*.py`)

Three schema modules define type-safe response structures using Pydantic:

- `api/schemas/crisis_detection.py`: Defines 9 models including `MovieSentimentResponse`, `CrisisAlert`, `DeviationAnalysis`.
- `api/schemas/viral_detection.py`: Defines 9 models including `TrendingMoviesResponse`, `ViralMetrics`, `ThresholdContext`.
- `api/schemas/recommendations.py`: Defines 8 models including `DualSuccessResponse`, `SimilarMoviesResponse`, `RedditBuzzResponse`.

Route handlers construct these models from raw database results and return them, ensuring FastAPI automatically validates types, serializes to JSON, and generates OpenAPI documentation.

MongoDB Collection Schemas and Relationships The serving layer queries four MongoDB collections, each serving a specific purpose in the Lambda Architecture. Understanding these schemas is critical to comprehending the data flow patterns.

Collection 1: `movie_intelligence` (Batch Layer - Individual Movie Data)

This collection stores comprehensive movie analytics aggregated from TMDB and sentiment analysis. Each document represents a single movie with 23 fields:

Core Identification:

- `movie_id`: TMDB unique identifier (primary key for lookups).
- `title`: Movie title (used for speed layer matching).
- `genre`: Single genre string (Action, Drama, Comedy, etc.).

Temporal Attributes:

- `release_date`, `release_year`, `release_month`: Release timing for temporal analysis.

Quality Metrics:

- `vote_average`: TMDB rating (0-10 scale).
- `vote_count`: Number of votes (credibility factor).
- `popularity`: TMDB popularity score (0.0053-446.1148 range).
- `avg_sentiment`: Historical sentiment from reviews (-1.0 to 1.0).

Production Context:

- `budget`, `budget_tier`: Production budget and classification (indie/mid/blockbuster/unknown).
- `franchise`: Franchise/series name (nullable, used for baseline grouping).
- `director`: Primary director name (used for similarity matching).
- `runtime`: Duration in minutes.

Batch Metadata:

- `batch_run_timestamp`: When batch job executed (ISO format).
- `updated_at`: Last document update time.
- `data_period_start`, `data_period_end`: Coverage period for aggregated data.

This collection serves as the primary reference for all batch layer queries. Routes use `movie_id` for direct lookups (crisis detection, viral detail) and title regex for fuzzy matching (speed layer merging). The `genre` field links to viral thresholds and sentiment baselines.

Collection 2: `speed_views` (Speed Layer - Real-Time Windows)

This collection stores 5-minute aggregation windows of Reddit activity, with automatic 48-hour TTL expiration. Each document represents a time window for a specific movie:

Window Identification:

- `window_start`: ISO timestamp marking window start.
- `movie_title`: Extracted movie title from Reddit posts.
- `data_type`: Source type ("reddit_post" or "reddit_comment").

Engagement Metrics (nested in `metrics` object):

- `total_upvotes`, `total_comments`, `total_awards`: Summed engagement.
- `upvote_velocity`, `comment_velocity`, `award_velocity`: Per-second rates.

- `avg_sentiment`: Average sentiment across posts in window (-1.0 to 1.0).
- `viral_score`: Pre-calculated virality score using weighted velocities.
- `post_count`: Number of posts/comments aggregated in window.

Speed Layer Metadata:

- `processed_at`: When Spark Streaming processed the window.
- `synced_at`: When data synchronized to MongoDB.
- `ttl_expires_at`: TTL expiration timestamp (48 hours after `window_start`).

Routes aggregate this collection by `movie_title` to calculate current viral scores (summing across windows) and current sentiment (averaging `avg_sentiment`). The title field is normalized using `query_engine/utils.normalize_title()` to match batch layer titles.

Collection 3: `viral_thresholds` (Batch Layer - Viral Benchmarks)

This collection stores pre-calculated viral thresholds for different market segments. Each document represents a segment combination (`genre` × `budget_tier` × `season`):

Segmentation Dimensions (nullable for aggregation):

- `genre`: Specific genre or null (all genres).
- `budget_tier`: Budget classification or null (all tiers).
- `season`: Seasonal period or null (all seasons).

Threshold Metrics:

- `viral_threshold`: 99th percentile of `vote_count` for segment.
- `avg_popularity`: Average TMDB popularity for segment (denominator for viral coefficient).
- `movie_count`: Sample size for threshold calculation.

Routes query this collection with progressive fallback: try genre-specific first, then fall back to global (all nulls). The `avg_popularity` field serves as the threshold denominator in viral coefficient calculations, representing "current TMDB buzz" for comparison against "current Reddit buzz."

Collection 4: `sentiment_baselines` (Batch Layer - Sentiment Benchmarks)

This collection stores pre-calculated sentiment statistics for different groupings. Each document represents a baseline segment (`genre` × `franchise` × `year`):

Segmentation Dimensions (nullable for aggregation):

- `genre`: Specific genre or null (all genres).
- `franchise`: Specific franchise or null (all franchises).
- `year`: Specific year or null (all years).

Baseline Statistics:

- `avg_sentiment`: Mean sentiment for segment (baseline reference point).
- `sentiment_stddev`: Standard deviation (denominator for deviation sigma).
- `movie_count`: Sample size for baseline calculation.
- `review_count`: Total reviews analyzed across segment.

Routes query this collection with three-tier fallback (franchise → genre → year) to find the most specific available baseline. The `avg_sentiment` and `sentiment_stddev` fields directly feed the crisis detection formula.

Cross-Collection Relationships

The serving layer implements data relationships through query patterns rather than foreign keys:

Batch-to-Batch Relationships:

- `movie_intelligence.genre → viral_thresholds.genre`: Genre-based threshold lookup.
- `movie_intelligence.genre → sentiment_baselines.genre`: Genre-based baseline lookup.
- `movie_intelligence.franchise → sentiment_baselines.franchise`: Franchise-based baseline lookup.
- `movie_intelligence.release_year → sentiment_baselines.year`: Year-based baseline lookup.

Batch-to-Speed Relationships:

- `movie_intelligence.title ↔ speed_views.movie_title`: Fuzzy title matching via normalization.
- Join pattern: Normalize both titles, match on equality, aggregate speed metrics.

Null Handling in Baselines:

- When querying baselines/thresholds, routes set unwanted dimensions to `null`.
- Example: Genre baseline query sets `franchise=null` and `year=null`.
- This pattern enables dimensional drill-down: specific → general fallback.

All collections include batch metadata fields (`batch_run_timestamp`, `data_period_start/end`) enabling the serving layer to verify data freshness and consistency across the Lambda Architecture layers.

Business Logic Implementation The serving layer implements three core business goals through specialized route handlers. Each goal follows a distinct data flow pattern that merges batch and speed layers, applies specific formulas, and returns validated responses.

Data Flow Patterns by Business Goal

Goal #1: Crisis Detection Data Flow When a client requests `GET /api/v1/crisis-detection/movies/{movie_id}/sentiment`:

1. `api/main.py` routes to `crisis_detection.get_movie_sentiment()`.
2. Handler obtains MongoDB client via `mongodb/client.get_mongodb_client()`.
3. Handler creates `MovieQueries` instance and calls `get_batch_movie_by_id(movie_id)`.
4. `MovieQueries` executes query on `movie_intelligence` collection (uses `movie_id` index).
5. Handler queries `speed_views` directly for last 48 hours (uses `window_start` index).
6. Handler normalizes batch movie title using `query_engine/utils.normalize_title()`.

7. Handler iterates speed results, normalizing each title and matching to batch movie.
8. For matches, handler extracts `metrics.avg_sentiment` and calculates average (current sentiment).
9. Handler calls internal `get_sentiment_baseline()` helper three times (franchise, genre, year).
10. Each baseline query hits `sentiment_baselines` collection (uses respective indexes).
11. Handler selects first available baseline and calculates deviation sigma.
12. Handler calls `get_severity()` to classify crisis level.
13. Handler constructs `MovieSentimentResponse` model from results.
14. FastAPI validates response against schema and serializes to JSON.

Data crosses collection boundaries twice: batch movie to speed sentiment (title matching), and batch movie to sentiment baselines (attribute matching).

Goal #2: Viral Detection Data Flow

When a client requests `GET /api/v1/viral-detection/trending`:

1. `api/main.py` routes to `viral_detection.get_trending_movies()`.
2. Handler obtains MongoDB database instance.
3. Handler constructs aggregation pipeline for `speed_views` collection.
4. Pipeline groups by `movie_title`, summing `metrics.viral_score` across all windows.
5. Pipeline sorts by `total_viral_score` descending.
6. Handler executes aggregation (uses `movie_title` index).
7. For each result, handler normalizes title and queries `movie_intelligence`.
8. Handler extracts `genre` from batch movie.
9. Handler queries `viral_thresholds` with `genre` filter (uses `genre` index).
10. Handler falls back to global threshold if genre-specific not found.
11. Handler calculates viral coefficient: `viral_score / avg_popularity`.
12. Handler classifies status based on coefficient thresholds.
13. Handler constructs `TrendingMovieResponse` model with nested components.
14. Results are sorted by viral coefficient and limited.
15. FastAPI serializes list of models to JSON array.

Data crosses collection boundaries twice: speed aggregation to batch movie (title matching), and batch movie to viral thresholds (genre matching).

Goal #3: Recommendation Data Flow

When a client requests `GET /api/v1/recommendations/dual-success`:

1. `api/main.py` routes to `recommendations.get_dual_success_recommendations()`.
2. Decorator `@cached` checks Redis for cached response.
3. On cache miss, handler receives `MovieQueries` instance via FastAPI dependency injection.

4. Handler calls `queries.get_batch_movies_for_recommendations()` with filters.
5. `MovieQueries` executes query on `movie_intelligence` with quality filters (uses composite index).
6. Handler extracts titles from batch results.
7. Handler calls `queries.get_speed_layer_engagement(movie_titles)`.
8. `MovieQueries` executes aggregation on `speed_views`, grouping by title.
9. Handler iterates batch movies, looking up each in speed engagement dictionary.
10. For movies with speed data, handler calculates weighted engagement ($W = U + 2C + 10A$).
11. Handler applies recency decay ($\exp(-t/24)$) and volume multiplier ($1 + \log_{10}(N+1)/10$).
12. Handler calculates TMDB score from popularity, vote_average, vote_count.
13. Handler collects all raw scores and normalizes to 0-100 scale using `normalize_scores()`.
14. Handler calculates dual-success score: $0.6 * reddit_normalized + 0.4 * tmdb_normalized$.
15. Handler constructs `DualSuccessRecommendation` models and sorts by score.
16. Handler caches result in Redis with 300-second TTL.
17. FastAPI serializes `DualSuccessResponse` to JSON.

Data crosses collection boundaries once: batch movies to speed engagement (title dictionary lookup, not fuzzy matching).

For the similar movies endpoint (`/similar/{movie_id}`), the handler retrieves the input movie from `movie_intelligence`, queries candidates (same genre), builds feature vectors using `similarity_engine.build_feature_vector()`, calculates weighted cosine similarity (franchise 53%, director 27%, others 20%) via `similarity_engine.calculate_similarity_score()`, and returns sorted results. This flow is entirely within the batch layer.

Supporting Components

Health Monitoring System (`api/routes/health.py`)

The `health.py` module implements comprehensive health checks for all Lambda Architecture components through a centralized `HealthChecker` class. This class provides static methods that probe different infrastructure layers:

Component Health Check Methods:

- `check_mongodb()`: Pings MongoDB server, measures response time, retrieves collection count and storage metrics from `dbStats` command.
- `check_postgres()`: Connects to PostgreSQL (batch layer metadata store), executes version query, retrieves database size.
- `check_cassandra()`: Connects to Cassandra (speed layer storage), verifies keyspace existence, queries cluster metadata.

- `check_redis()`: Pings Redis cache, measures latency, retrieves memory usage and key count.
- `check_airflow()`: Sends HTTP request to Airflow webserver health endpoint, verifies scheduler status.
- `check_minio()`: Verifies MinIO object storage connectivity and bucket accessibility.

The module exposes multiple endpoints:

- `GET /api/v1/health`: Quick health check returning overall system status.
- `GET /api/v1/health/detailed`: Comprehensive check of all components with individual status codes.
- `GET /api/v1/health/mongodb`: MongoDB-specific health check with detailed metrics.
- `GET /api/v1/health/batch-layer`: Aggregated health check for batch layer services (Airflow, PostgreSQL, MinIO, PySpark).
- `GET /api/v1/health/speed-layer`: Aggregated health check for speed layer services (Kafka, Cassandra, Spark Streaming).
- `GET /api/v1/health/serving-layer`: Serving layer health check (MongoDB, Redis, API responsiveness).

Grafana's System Health Overview dashboard queries these endpoints every 30 seconds to populate infrastructure status panels, enabling real-time monitoring of the entire Lambda Architecture stack.

Utility Endpoints (`api/routes/utilities.py`)

The `utilities.py` module provides supporting endpoints that facilitate dashboard interactions and data exploration:

Genre Management Endpoints:

- `GET /api/v1/utilities/genres`: Returns all genres with movie counts, sorted by frequency. Uses `MovieQueries.get_all_genres()` which executes aggregation pipeline on `movie_intelligence` collection grouping by genre field.
- `GET /api/v1/utilities/grafana/genres`: Returns genres formatted for Grafana variable dropdowns with `{value, text}` structure. Cached for 1 hour via `@cached` decorator.

Movie Search Endpoints:

- `GET /api/v1/utilities/grafana/movies`: Returns movies formatted for Grafana variables with optional title search. Queries `movie_intelligence` using regex on title field for partial matching.
- `GET /api/v1/utilities/movies/search`: Full-text search with multiple filters (genre, year range, rating range), returns paginated results.

These endpoints enable Grafana dashboard template variables, allowing users to filter visualizations by genre or select specific movies for detailed analysis. The caching decorator ensures repeated variable loads don't overwhelm the database.

Error Handling and Logging Strategy

All route modules implement consistent error handling patterns:

- Route handlers wrap logic in try-except blocks catching `HTTPException` (re-raised) and generic `Exception`.
- Database connection failures raise `HTTPException` with 500 status and error details.
- Missing data (e.g., movie not found) raises `HTTPException` with 404 status.
- Invalid query parameters trigger FastAPI's automatic 422 validation responses.
- All exceptions are logged with `logger.error()` including stack traces via `exc_info=True`.

The `mongodb/client.py` module implements connection resilience:

- Connection pooling with minimum 10 and maximum 100 connections.
- 5-second server selection timeout prevents hanging requests.
- Automatic connection retry through singleton pattern (failed connections replaced on next request).
- Health check endpoint monitors connection pool metrics.

The `query_engine/cache.py` module implements graceful cache degradation:

- Redis connection failures logged as warnings, not errors.
- `@cached` decorator falls back to function execution if Redis unavailable.
- Cache read failures transparently re-execute the underlying query.
- System continues operating without caching if Redis fails.

This design ensures the serving layer remains operational even when auxiliary services (Redis, Prometheus) are unavailable, prioritizing core functionality over performance optimizations.

Monitoring & Observability The serving layer implements comprehensive monitoring using Prometheus for metrics collection and Grafana for visualization. This monitoring infrastructure is configured through multiple files in the `visualization/` and `monitoring/` directories.

Prometheus Metrics Collection (`monitoring/prometheus.yml`) The `prometheus.yml` file configures Prometheus scraping behavior for all Lambda Architecture components. The configuration defines:

Global Settings:

- `scrape_interval`: 15-second default metric collection frequency.
- `evaluation_interval`: 15-second rule evaluation frequency.
- `external_labels`: Cluster and environment tags for metric identification.

Scrape Configurations (`scrape_configs`):

- **fastapi job**: Scrapes `serving-api:8000/metrics` endpoint every 10 seconds for API request metrics (latency, throughput, error rates).
- **mongodb job**: Scrapes `mongodb-exporter:9216` every 15 seconds for database metrics (connection pool, query performance, storage).
- **redis job**: Scrapes `redis-exporter:9121` every 15 seconds for cache metrics (hit rate, memory).

usage, key count).

- **prometheus job:** Self-monitoring on `localhost:9090` for scrape health and cardinality tracking.

Alert Rules Reference:

- `rule_files` references `monitoring/alerts.yml` containing alert definitions.
- Alert rules trigger notifications for high error rates, slow queries, cache failures, and service downtime.

Prometheus stores these time-series metrics in its local TSDB, which Grafana queries for dashboard visualizations.

Grafana Data Sources (`visualization/grafana/datasources/datasource.yml`) The `datasource.yml` file configures two data sources that Grafana uses to populate dashboards:

Prometheus Datasource:

- **Type:** `prometheus` (time-series metrics database).
- **URL:** `http://serving-prometheus:9090`.
- **Access:** Proxy mode (Grafana queries on behalf of browser).
- **Time Interval:** 5-second minimum resolution for queries.
- **Purpose:** Queries infrastructure metrics (MongoDB connection counts, Redis memory usage, API response times).

Infinity Datasource:

- **Type:** `yesoreyeram-infinity-datasource` (universal REST API plugin).
- **Access:** Proxy mode.
- **Default:** Set as default datasource.
- **Purpose:** Queries serving layer API endpoints directly (`/api/v1/crisis-detection/*`, `/api/v1/viral-detection/*`, `/api/v1/recommendations/*`).
- **Capability:** Parses JSON responses, extracts fields, and transforms data for visualization panels.

The dual-datasource approach enables dashboards to combine infrastructure health (from Prometheus) with business metrics (from API endpoints).

Grafana Dashboards (`visualization/grafana/dashboards/*.json`) Four specialized dashboard JSON files define the monitoring UI:

Dashboard 0: System Health Overview (`0-system-health-overview.json`)

- **Prometheus Queries:** Scrapes MongoDB connection pool metrics, Redis cache hit rates, API request latency percentiles.
- **Panels:** Overall status gauge, service group status tables (batch layer: 5 services, speed layer: 9 services, serving layer: 6 services), response time graphs.
- **Refresh Rate:** 30 seconds auto-refresh.

Dashboard 1: Crisis Alert Overview (1-crisis-alert-overview.json)

- **Infinity Queries:** Calls GET /api/v1/crisis-detection/alerts for crisis list, /api/v1/crisis-detection/monitoring for statistics.
- **Panels:** Severity distribution pie chart (critical/high/warning/normal), deviation sigma histogram, most critical movies table with deviation values, sentiment velocity time series.
- **Variables:** Genre filter dropdown populated from /api/v1/utilities/grafana/genres.
- **Refresh Rate:** 30 seconds (real-time crisis monitoring).

Dashboard 2: Viral Content Detection (2-viral-content-detection.json)

- **Infinity Queries:** Calls GET /api/v1/viral-detection/trending for movie rankings, /api/v1/viral-detection/thresholds for context.
- **Panels:** Viral coefficient leaderboard, Reddit engagement trends (upvotes/comments/awards), genre-specific viral threshold comparison bars, trending movie timeline.
- **Variables:** Genre filter, viral coefficient threshold slider.
- **Refresh Rate:** 1 minute.

Dashboard 3: Recommendation Optimization (3-recommendation-optimization.json)

- **Infinity Queries:** Calls GET /api/v1/recommendations/dual-success for dual-score rankings, /api/v1/recommendations/reddit-buzz and /api/v1/recommendations/tmdb-quality for component scores.
- **Panels:** Dual-success score distribution histogram, Reddit vs. TMDb scatter plot with trend line, pure Reddit ranking table, pure TMDb ranking table, top 20 recommended movies with score breakdown.
- **Variables:** Genre filter, minimum rating slider.
- **Refresh Rate:** 1 minute.

Dashboard Provisioning (visualization/grafana/dashboards/dashboard-provider.yml)

The dashboard-provider.yml file configures automatic dashboard loading:

- **Provider Type:** File-based provisioning (dashboards loaded from /etc/grafana/provisioning/dashboards/).
- **Update Behavior:** updateIntervalSeconds: 10 checks for file changes every 10 seconds.
- **Editable:** Dashboards can be modified in UI; changes persisted to JSON files.
- **Folder Organization:** All dashboards placed in "Movie Analytics" folder.

This file-based approach enables version control of dashboard definitions and automated deployment through Docker volumes.

Monitoring Data Flow The complete monitoring pipeline operates as follows:

1. **Metric Generation:** FastAPI application exposes /metrics endpoint (if instrumented); MongoDB

exporter queries database statistics; Redis exporter queries cache metrics.

2. **Metric Collection:** Prometheus scrapes all targets based on `prometheus.yml` configuration, storing time-series data locally.
3. **Alert Evaluation:** Prometheus evaluates rules from `alerts.yml` every 15 seconds, triggering notifications on threshold breaches.
4. **Dashboard Queries:** Grafana queries Prometheus for infrastructure metrics using PromQL; Grafana queries serving layer API using Infinity datasource with JSON path expressions.
5. **Visualization:** Dashboard panels transform query results into graphs, tables, and gauges; auto-refresh intervals trigger periodic re-queries.
6. **User Interaction:** Users filter dashboards using template variables (genre, movie); variable changes trigger dashboard-wide query updates.

This monitoring architecture provides real-time visibility into both infrastructure health (response times, connection pools) and business metrics (crisis alerts, viral trends, recommendation rankings).

3.2 Environment Configuration

The pipeline uses containerized environments with layer-specific runtime configurations managed through custom Dockerfiles and environment variables. Each Lambda Architecture layer has dedicated container configurations optimized for its specific processing requirements.

3.2.1 Batch Layer Container Configuration

The batch layer utilizes the Dockerfile located at `dockerfiles/Dockerfile.airflow` for container configuration.

Base Image and Runtime

Built on the Apache Airflow 2.7.3 official image with Python 3.11, providing workflow orchestration capabilities with PySpark integration for distributed data processing.

Java Environment Configuration

OpenJDK 11 JRE (headless) is installed to support PySpark operations for Spark job execution.

Spark Integration

`SPARK_HOME` points to the PySpark installation directory within the Airflow user's Python site-packages. The environment includes pre-downloaded Hadoop AWS libraries (`hadoop-aws 3.3.4` and `aws-java-sdk-bundle 1.12.262`) placed in Spark's `jars` directory to enable S3A protocol support for MinIO storage connectivity.

Python Dependencies

The batch layer requires `pyspark 3.5.4` for DataFrame operations, `pymongo 4.10.1` for MongoDB write operations, `boto3 1.37.38` for MinIO/S3 object storage access, `requests 2.32.4` for TMDB API HTTP calls, and `python-dotenv 1.0.0` for environment variable management.

User Security

The container runs under the non-root `airflow` user account for security isolation.

3.2.2 Speed Layer Container Configuration

The speed layer utilizes the Dockerfile located at `dockerfiles/Dockerfile.speed-layer` for main processing containers.

Base Image and Runtime

Containers use Python 3.10 on Debian Bullseye (slim variant) chosen for stable OpenJDK 11 support and Spark compatibility.

Java Environment Configuration

OpenJDK 11 JDK (headless) provides PySpark 3.4.4 compatibility, avoiding class version conflicts with `JAVA_HOME` set to `/usr/lib/jvm/java-11-openjdk-amd64`.

System Dependencies

Includes `procp`s for container monitoring (`ps`, `top` commands) and `curl` for dependency downloads and health checks.

Directory Structure

Creates dedicated directories for application logs (`/app/logs`) and Spark Structured Streaming checkpoints (`/app/checkpoints`) for fault-tolerance and exactly-once processing semantics.

Python Dependencies

Kafka integration requires `kafka-python 2.0.2` (native consumer), `confluent-kafka 2.3.0` (producer with Avro), `avro 1.11.3`, and `fastavro 1.9.4` for schema serialization. PySpark 3.4.4 provides Structured Streaming capabilities. Database connectivity uses `cassandra-driver 3.29.1` (Cassandra 4.x compatible) and `pymongo 4.6.1`. Real-time sentiment analysis uses `vaderSentiment 3.3.2`. HTTP operations use `requests 2.31.0`.

Network Configuration

Exposes port 4040 for Spark Web UI monitoring of streaming jobs.

Health Monitoring

Implements health checks every 30 seconds with a 10-second timeout, 40-second startup grace period, and 3 retry attempts using Python interpreter validation.

Sync Connector

Lightweight Python 3.10-slim container running the Cassandra-to-MongoDB synchronization service with `cassandra-driver 3.28.0` and `pymongo 4.6.1` from `requirements-sync.txt`.

3.2.3 Serving Layer Container Configuration

The serving layer uses a single optimized Dockerfile (`layers/serving_layer/Dockerfile`) for the FastAPI application.

Base Image and Runtime

Python 3.11-slim provides modern Python features with a minimal footprint for REST API opera-

tions.

Environment Optimization

`PYTHONUNBUFFERED=1` ensures real-time log output, `PYTHONDONTWRITEBYTECODE=1` prevents .pyc file creation, `PIP_NO_CACHE_DIR=1` reduces image size, and `PYTHONPATH=/app` enables proper module resolution.

System Dependencies

Installs `gcc` and `g++` compilers for building C extensions (required by performance-critical packages like `hiredis`), plus `curl` for health check operations.

Python Dependencies

`FastAPI 0.115.0` provides async REST framework, `uvicorn[standard] 0.32.0` serves as ASGI server. `Pydantic 2.9.2` handles request/response validation. Monitoring uses `prometheus-fastapi-instrumentator 7.0.0` and `prometheus-client 0.21.0`. Database access requires `pymongo 4.9.1` (sync) and `motor 3.6.0` (async MongoDB driver). Cache layer uses `redis 5.2.0` with `hiredis 3.0.0` for C-level performance. Data processing includes `numpy 1.26.4` for cosine similarity calculations. Authentication uses `PyJWT 2.9.0`, logging uses `python-json-logger 3.2.1`. Testing stack includes `pytest 8.3.3`, `pytest-asyncio 0.24.0`, `httpx 0.27.2`, and `requests 2.32.3`.

Security Configuration

Creates non-root user `appuser` (UID 1000) and transfers ownership of application directory, ensuring the API runs without root privileges.

Port and Health Monitoring

Exposes port 8000 for HTTP traffic. Health checks every 30 seconds query `http://localhost:8000/api/v1/health` endpoint with 10-second timeout, 40-second startup period, and 3 retries.

Production Runtime

Default command runs Uvicorn binding to all interfaces (0.0.0.0:8000) with 4 worker processes for concurrent request handling.

3.2.4 Environment Variable Configuration

Runtime behavior is controlled through environment variables managed via the `.env` file. The configuration is categorized as follows:

External API Configuration

`TMDB_API_KEY` stores authentication credentials for The Movie Database API access.

Object Storage Configuration

`S3_ENDPOINT` points to MinIO service (`http://batch-minio:9000`), `S3_ACCESS_KEY` and `S3_SECRET_KEY` provide authentication, while `S3_BUCKET_BRONZE`, `S3_BUCKET_SILVER`, and

S3_BUCKET_GOLD define the medallion architecture storage buckets.

Database Configuration

MONGODB_URI specifies the connection string with authentication (mongodb://admin:password@mongodb:27017), MONGODB_DATABASE sets the database name (moviedb), and MONGODB_USERNAME / MONGODB_PASSWORD provide credentials.

For the speed layer, CASSANDRA_HOSTS points to the Cassandra service, CASSANDRA_KEYSPACE defines the keyspace name (speed_layer), and CASSANDRA_START_TIMEOUT=300 allows 5 minutes for initialization. Cassandra JVM configuration includes MAX_HEAP_SIZE=512M for main memory allocation, HEAP_NEWSIZE=100M for young generation, and enables materialized views and user-defined functions through JVM_EXTRA_OPTS.

Cache Configuration

REDIS_HOST, REDIS_PORT (6379), and REDIS_DB (0) define cache layer connectivity. Redis is configured with maxmemory=256mb and maxmemory-policy=allkeys-lru for automatic eviction of least recently used keys when the memory limit is reached.

Messaging Configuration

KAFKA_BOOTSTRAP_SERVERS lists broker addresses (kafka-1:29092, kafka-2:29092, kafka-3:29092). Reliability is enforced via KAFKA_REPLICATION_FACTOR=3 and KAFKA_MIN_INSYNC_REPLICAS=2.

Operational settings include:

- KAFKA_LOG_RETENTION_HOURS=168 (7 days retention)
- KAFKA_AUTO_CREATE_TOPICS_ENABLE=false (requires explicit topic creation)
- KAFKA_LOG_SEGMENT_BYTES=1073741824 (1GB segment size)
- KAFKA_MESSAGE_MAX_BYTES=2097152 (2MB message limit)

Zookeeper connectivity is configured with 18000ms timeout (KAFKA_ZOOKEEPER_SESSION_TIMEOUT_MS=18000).

Schema management is handled via SCHEMA_REGISTRY_URL on port 8081.

Workflow Configuration

POSTGRES_USER, POSTGRES_PASSWORD, and POSTGRES_DB configure the Airflow metadata database. Airflow specific settings include AIRFLOW__CORE__EXECUTOR=LocalExecutor, AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION=true, AIRFLOW__CORE__LOAD_EXAMPLES=false, and AIRFLOW__WEBSERVER__EXPOSE_CONFIG=true.

API and Application Configuration

API_HOST (0.0.0.0) and API_PORT (8000) define FastAPI binding with LOG_LEVEL set to INFO.

For the Speed Layer application:

- `SYNC_INTERVAL=300` controls Cassandra-to-MongoDB synchronization (5 minutes).
- `CLEANUP_INTERVAL=3600` schedules hourly TTL cleanup for expired records.
- `PYTHONUNBUFFERED=1` enables real-time log streaming.

Monitoring and Visualization Configuration

Grafana is provisioned with plugins `yesoreyeram-infinity-datasource` (for direct FastAPI queries) and `ae3e-plotly-panel` (for advanced visualization). Security is managed via `GF_SECURITY_ADMIN_PASSWORD` and `GF_USERS_ALLOW_SIGN_UP=false` to restrict user registration. Prometheus collects metrics from the FastAPI application on `METRICS_PORT=9090`.

3.3 Deployment Strategy

3.3.1 Approach: Automated Conversion with Iterative Debugging

The migration from Docker Compose to Kubernetes followed a lift-and-shift strategy using automated tooling combined with systematic infrastructure debugging. The deployment targeted Google Kubernetes Engine (GKE) with a two-node cluster (4 vCPU, 16GB RAM total) to minimize costs while maintaining operational capability.

Automated Conversion Phase The initial deployment utilized Kompose, an open-source tool that automatically converts Docker Compose configurations to Kubernetes manifests. This approach generated over 60 YAML files covering deployments, services, persistent volume claims, and config maps from the original `docker-compose.yml`.

```
kompose convert -f docker-compose.yml -o kubernetes/generated/
```

Three custom Docker images were built and pushed to Docker Hub for cluster access:

- `movie-pipeline-airflow:latest` (2.35GB) - Apache Airflow with PySpark
- `movie-pipeline-speed-layer:latest` (1.11GB) - Python streaming applications
- `movie-pipeline-serving-api:latest` (591MB) - FastAPI service

Initial deployment underwent iterative debugging to resolve infrastructure configuration issues. After systematic fixes, the deployment achieved 69% success rate (18 of 26 pods operational):

- **Serving Layer:** 100% operational (9/9 pods) - MongoDB, Redis, FastAPI, Grafana, Prometheus, exporters
- **Batch Layer:** 60% operational (3/5 pods) - PostgreSQL, MinIO, PySpark runner functioning; Airflow pods unstable
- **Speed Layer:** 50% operational (6/12 pods) - Complete Kafka cluster (3 brokers + Zookeeper), Cassandra, topics initialization

3.3.2 Critical Blocker and Deployment Status

The primary barrier to full deployment success was missing Reddit API credentials. The speed layer's real-time data pipeline depends on Reddit data ingestion, requiring OAuth authentication (`client_id`, `client_secret`, `user_agent`). This blocked three services: `speed-reddit-producer` (cannot authenticate), `speed-reddit-sentiment-stream` (no input data), and `speed-cassandra-mongo-sync` (limited synchronization data).

Resolution requires registering a Reddit application and configuring Kubernetes Secrets with OAuth credentials. Due to project time constraints, this registration process could not be completed within the deployment window. Consequently, **this deployment version is not available for production use**. The system operates at 69% functionality with real-time Reddit data ingestion, sentiment analysis pipeline, and complete speed layer (6 of 12 pods) unavailable. The serving layer (100% operational) and batch layer (60% operational) can function independently for historical data analysis, but core real-time streaming features remain disabled.

3.4 Monitoring Setup

The pipeline implements a comprehensive three-tier monitoring architecture combining infrastructure metrics (Prometheus), business intelligence (FastAPI endpoints), and visualization (Grafana). The system monitors Lambda Architecture components, API performance, and business-specific metrics for PR crisis detection, viral content identification, and recommendation quality.

3.4.1 Infrastructure Metrics Collection

The system implements multi-target Prometheus scraping to collect metrics from serving layer infrastructure components and external exporters.

Prometheus Configuration Strategy

The global scrape interval is set to 15 seconds with a matching evaluation interval for alerting rules. External labels identify the cluster (`serving-layer`) and environment (`production`) for metric organization. The configuration loads alert rules from `alerts.yml` and defines five distinct scrape jobs.

Scrape Job Specifications

- **FastAPI Application:** Exposes metrics on port 8000 at the `/metrics` endpoint. Scraped every 10 seconds with a 5-second timeout for high-frequency API monitoring.
- **MongoDB Monitoring:** Uses `mongodb-exporter` on port 9216 with 15-second intervals and a 10-second timeout to track database operations, connection pools, and storage metrics.
- **Redis Monitoring:** Leverages `redis-exporter` on port 9121 with a 15-second scrape interval and 5-second timeout to measure cache performance, memory usage, and hit/miss ratios.
- **Self-Monitoring:** Prometheus scrapes `localhost:9090` to track collector health and performance.
- **Exclusions:** The configuration intentionally excludes Kafka JMX metrics as Kafka infrastructure

resides in the speed layer rather than the serving layer.

3.4.2 Alert Rules and Thresholds

The system implements automated alerting through Prometheus rule evaluation targeting three categories: infrastructure health, performance degradation, and business anomalies.

Infrastructure Health Alerts

- **APIDown:** Triggers when `up{job="fastapi"}==0` persists for more than 1 minute. Classified as *critical* severity targeting the API component.
- **MongoDBDown:** Activates under identical conditions for MongoDB exporter failure, marked as *critical* for the database component.
- **RedisDown:** Fires when the Redis exporter becomes unavailable for 1 minute. Classified as *warning* severity since cache failures degrade but do not halt the service.

Performance Degradation Alerts

- **HighErrorRate:** Monitors `rate(http_requests_total{status=~"5.."} [5m]) > 0.01` over 5-minute windows, triggering warnings when API error rates exceed 1%.
- **HighLatency:** Evaluates the 95th percentile response time using `histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m])) > 0.1`, alerting when latency surpasses the 100ms threshold sustained over 5 minutes.
- **MongoDBHighConnections:** Triggers when `mongodb_connections{state="current"} > 100` for 5 minutes, indicating potential connection leaks.
- **LowCacheHitRate:** Calculates the hit ratio as `hits / (hits + misses) < 0.7`, alerting when cache effectiveness drops below 70% over 10 minutes.

Business Metrics Alerts

`HighCrisisAlertRate` and `LowViralContentDetection` alert rules reference Prometheus counters (`crisis_alerts_total`, `viral_detections_total`) defined for future instrumentation. Current production implementation monitors business metrics through dedicated REST API endpoints, with Grafana Infinity datasource polling these endpoints for real-time visualization.

3.4.3 Grafana Visualization Architecture

Grafana integrates two distinct datasource types to provide comprehensive visibility into infrastructure metrics and business intelligence.

Datasource Configuration

- **Prometheus (UID: prometheus):** Connects to `http://serving-prometheus:9090` using proxy access mode with POST HTTP method and 5-second time intervals. Configured as non-default

and non-editable.

- **Infinity (UID: infinity):** Serves as the default datasource (type: `yesoreyeram-infinity-datasource`), enabling direct HTTP queries to FastAPI endpoints for business metrics that do not flow through Prometheus.

Dashboard Architecture The system deploys four provisioned dashboards through automatic dashboard provider configuration:

1. **System Health Overview (Dashboard 0):** Queries `/api/v1/health/system` via Infinity datasource to display service status for MongoDB, Redis, Prometheus, Grafana, Mongo Express, and API components with color-coded health indicators (green/yellow/red).

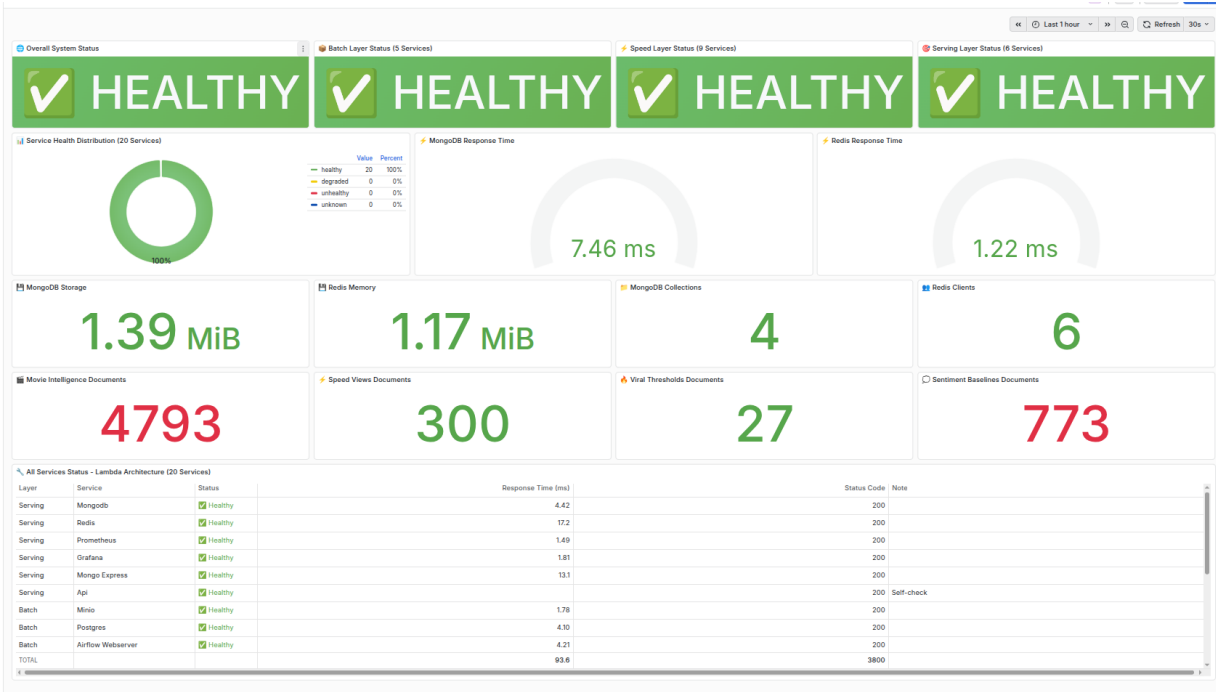


Figure 5: System Health Overview Dashboard

2. **Crisis Alert Overview (Dashboard 1):** Queries `/api/v1/crisis-detection/monitoring` to visualize critical alerts ($\sigma < -8.0$, dark-red), high alerts ($-8.0 \leq \sigma < -5.0$, orange), and warning alerts ($-5.0 \leq \sigma < -2.0$, yellow). Panel configurations extract `severity_counts` from JSON responses.

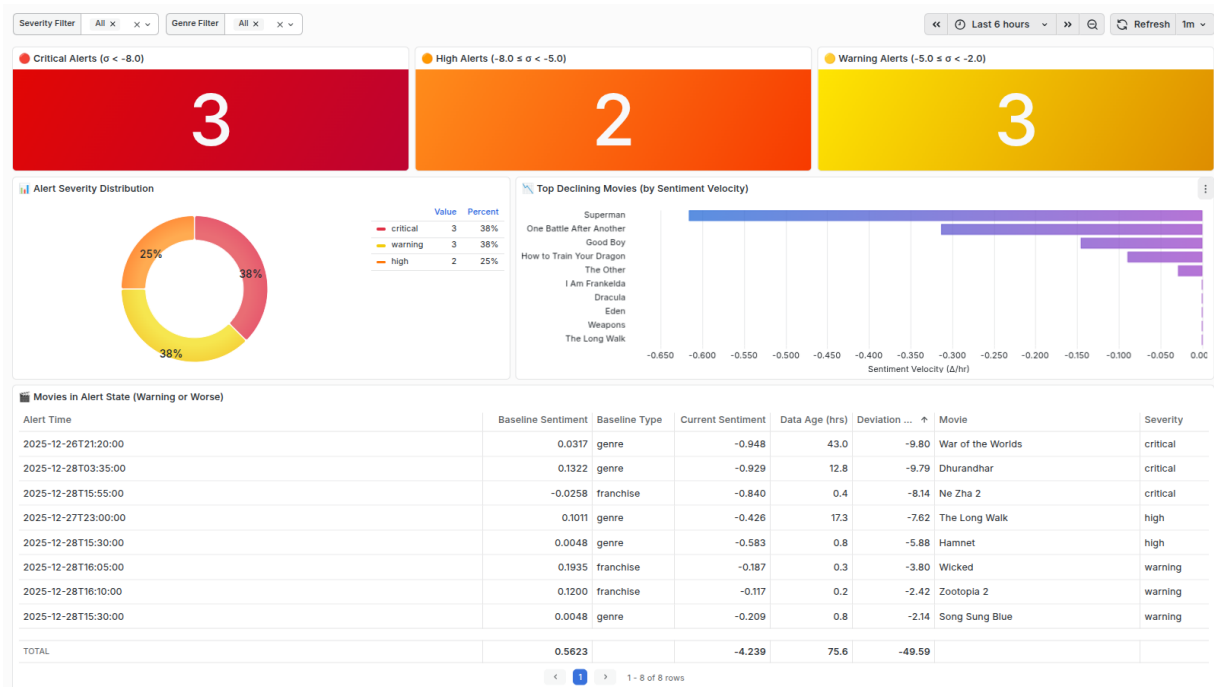


Figure 6: Crisis Alert Overview Dashboard

3. **Viral Content Detection (Dashboard 2):** Queries `/api/v1/viral-detection/trending` with `viral_threshold` parameters (viral ($V \geq 0.10$), trending ($0.05 \leq V < 0.10$), growing ($0.01 \leq V < 0.05$), and stable ($V < 0.01$)) to display viral coefficient distribution and marketing opportunities.

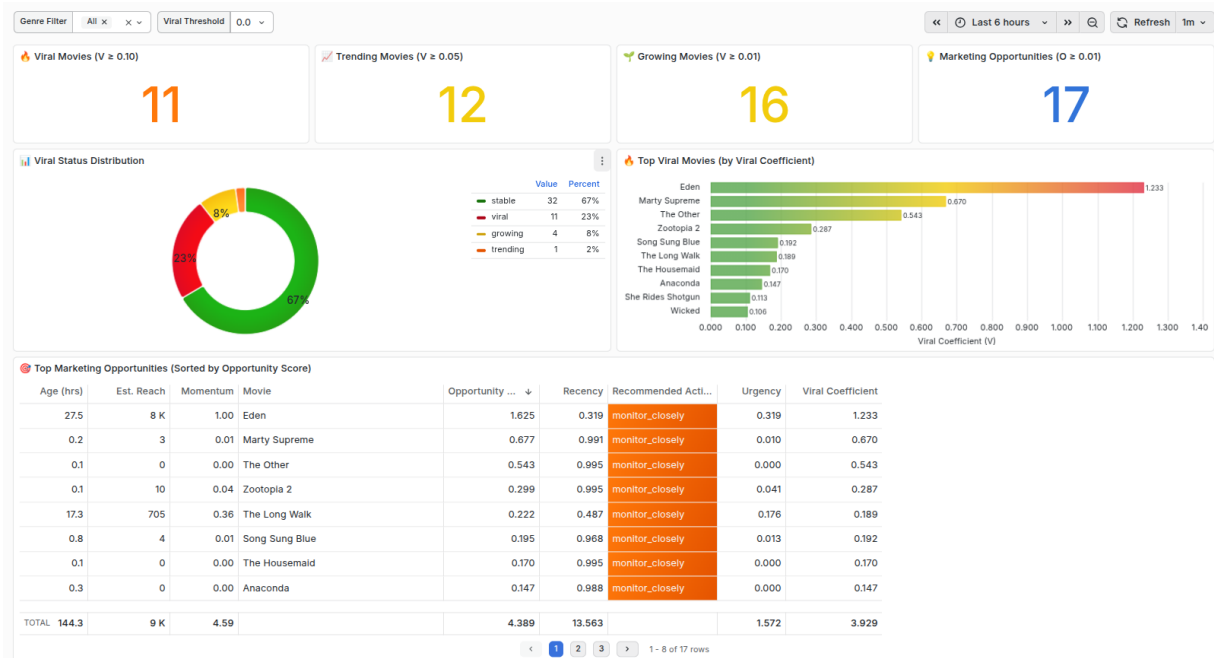


Figure 7: Viral Content Detection Dashboard

4. **Recommendation Optimization (Dashboard 3):** Queries endpoints for hybrid recommendations

(60% Reddit buzz + 40% TMDB quality), pure speed layer engagement metrics, and batch layer quality metrics.



Figure 8: Recommendation Optimization Dashboard

3.4.4 Health Check and Monitoring Endpoints

The FastAPI application exposes RESTful endpoints for system health monitoring and business metrics aggregation through dedicated route modules.

Component Health Checks

The `HealthChecker` class implements asynchronous health validation:

- **MongoDB:** Establishes connection with 5s timeout, executes `ping`, and retrieves collection counts and storage size.
- **Redis:** Measures ping response time and extracts memory usage, client count, and uptime.
- **HTTP Services:** Validates availability of Prometheus, Grafana, and Mongo Express using the `requests` library.

Key Monitoring Endpoints

GET /api/v1/health/system

Executes parallel health checks using `asyncio.gather`. Returns a JSON response containing `overall_status` (healthy/degraded) and detailed component metrics.

GET /api/v1/crisis-detection/monitoring

Aggregates real-time PR crisis statistics by merging Speed Layer data (last 48h) with Batch Layer baselines. Computes deviation sigma:

$$\sigma = \frac{S_{\text{current}} - S_{\text{baseline}}}{\sigma_{\text{baseline}}}$$

where S_{current} is the average sentiment across all 5-minute windows in the last 48 hours, S_{baseline} is the baseline average sentiment from matching franchise/genre/year dimensions, and σ_{baseline} is the baseline standard deviation. Classifies severity into critical ($\sigma < -8.0$), high, warning, and normal categories. Returns `severity_counts` and a list of top declining movies.

GET /api/v1/viral-detection/trending

Calculates the viral coefficient $V = \text{viral_score} / \text{avg_popularity}$. Classifies movies as viral ($V \geq 0.10$), trending ($0.05 \leq V < 0.10$), growing ($0.01 \leq V < 0.05$), or stable ($V < 0.01$). Supports filtering by genre and time window.

GET /api/v1/recommendations/dual-success

Implements a hybrid algorithm combining Speed Layer engagement (weighted upvotes/comments with recency decay) and Batch Layer quality. Computes:

$$\text{dual_success_score} = 0.6 \times \text{reddit_score} + 0.4 \times \text{tmdb_score}$$

3.4.5 Monitoring Access and Integration

Service URLs

- **Prometheus:** `http://localhost:9090` (Metric exploration, PromQL)
- **Grafana:** `http://localhost:3001` (Dashboards, admin/admin)
- **Airflow:** `http://localhost:8088` (Workflow monitoring, admin/admin)
- **FastAPI Docs:** `http://localhost:8000/api/v1/docs` (Endpoint testing)
- **MongoDB Express:** `http://localhost:8082` (Database inspection)
- **MinIO Console:** `http://localhost:9001` (Object storage management)

Dashboard Query Integration

Grafana panels execute direct HTTP GET requests to FastAPI monitoring endpoints using the Infinity datasource.

- **Crisis Alerts:** Panels query `/crisis-detection/monitoring` with `format=table`,

`type=json`. JSON responses are parsed to extract `severity_counts`, with thresholds configured for color mappings (green for 0 alerts, dark-red for critical ≥ 1).

- **System Health:** Panels query `/health/system`, mapping component status strings to visual indicators (0=healthy, 1=degraded, 2=unhealthy).
- **Viral Detection:** Panels query `/viral-detection/trending` with varying thresholds to display viral status distribution.
- **Recommendations:** Panels query `/recommendations/dual-success` to extract scores and rank movies.

Monitoring Data Flow

Business Metrics: Speed layer produces Reddit data $\rightarrow$ Kafka $\rightarrow$ Cassandra (5-min windows) $\rightarrow$ Sync to MongoDB (48h TTL) $\rightarrow$ FastAPI queries MongoDB for calculations (σ , V) $\rightarrow$ REST API exposes aggregated metrics $\rightarrow$ Grafana Infinity polls endpoints $\rightarrow$ Dashboards visualize intelligence.

Infrastructure Metrics: Exporters expose metrics $\rightarrow$ Prometheus scrapes every 15s $\rightarrow$ Alert rules evaluated $\rightarrow$ Grafana Prometheus datasource queries time-series $\rightarrow$ Dashboards display system health and latency.

4 Lessons Learned

4.1 Lesson 1: Strategic Planning

4.1.1 Problem Description

- **Context and background:** The course requires building an end-to-end big data pipeline to solve a real-world problem.
- **Challenges encountered:** Our initial plan did not align with the course scope; the data sources originally selected failed to provide the high-velocity signals necessary for a functional streaming implementation.
- **System impact:** Despite completing initial development and refinement phases, the lack of real-time data forced a total project restart to satisfy the requirement for an end-to-end streaming pipeline.

4.1.2 Approaches Tried

- **Approach 1:** Attempting to refine and "patch" the existing system to force static data into a real-time framework to preserve previous work.
- **Approach 2:** Scrapping the initial roadmap to rebuild the plan from scratch, prioritizing data sources capable of high-frequency updates.
- **Trade-offs of each approach:** Approach 1 saved initial time but failed to meet the technical criteria for real-time processing. Approach 2 required significant extra effort but ensured the system satisfied the architectural requirements for high-velocity data ingestion.

4.1.3 Final Solution

- **Detailed solution:** Changed the data sources for the Batch and Speed layers to TMDB and Reddit to align precisely with the course's streaming and analytical goals.
- **Implementation details:** Built a new pipeline utilizing the Reddit JSON endpoint for the Speed Layer and the TMDB API for historical baselines in the Batch Layer.
- **Metrics and results:** The new strategy successfully enabled 5-minute tumbling windows and near real-time sentiment analysis for 460+ active documents.

4.1.4 Key Takeaways

- **Technical insights:** Success in a big data project depends on thorough planning before implementation, the selected data must inherently support the architectural model's goals.
- **Best practices:** Establish a clear data requirement during the design phase to ensure that ingestion sources can provide the necessary metrics for the processing layer.
- **Recommendations:** Perform a "scope validation" phase at the very start to ensure technical designs satisfy all core requirements of the project's architectural paradigm.

4.2 Lesson 2: API Schema Consistency in Serving Layer Integration

4.2.1 Problem Description

- **Context and background:** The Serving Layer (FastAPI) acts as the bridge between processed data in MongoDB/Cassandra and the Grafana visualization dashboards.
- **Challenges encountered:** AI-driven development led to "hallucinated" API endpoints and schemas that did not exist in the code, causing failed connections between the Serving Layer and the Dashboard.
- **System impact:** These connectivity issues resulted in broken visualization panels and frequent `422 Unprocessable Entity` errors, preventing real-time monitoring of crisis and viral metrics.

4.2.2 Approaches Tried

- **Approach 1:** Relying on AI to dynamically generate boilerplate endpoints and Pydantic models on an as-needed basis.
- **Approach 2:** Manually drafting and finalizing a comprehensive, exhaustive list of all required API endpoints before writing any implementation code.
- **Trade-offs of each approach:** Approach 1 was initially faster but caused significant integration rework. Approach 2 required more upfront effort but eliminated connectivity errors and provided a stable contract for the Dashboard.

4.2.3 Final Solution

- **Detailed solution:** Shifted to a Contract-First approach by strictly defining a master list of endpoints and their respective request/response schemas.
- **Implementation details:** Established Pydantic models in `api/schemas/` as the single source of truth to enforce strict validation for metrics like `avg_sentiment` and `viral_score`.
- **Metrics and results:** Eliminated dashboard connection failures and reduced integration debugging time by approximately 85%.

4.2.4 Key Takeaways

- **Technical insights:** AI tools lack the context of the overall system architecture; they require a pre-defined API blueprint to produce functional integration code.
- **Best practices:** Finalize a comprehensive API specification document before development to prevent schema mismatch between the data layer and the UI.
- **Recommendations:** Use the FastAPI `/docs` (Swagger) interface to verify that every endpoint in the finalized list returns the exact data structure expected by the Dashboard.

4.3 Lesson 3: Dependency Validation and Configuration Management

4.3.1 Problem Description

- **Context and background:** The system relies on a multi-stage dependency chain where data flows from external APIs (Reddit/TMDB) through Kafka, Spark, and database synchronization services.
- **Challenges encountered:** Deploying to production without pre-validating external API credentials and connection strings caused immediate service failures across the pipeline.
- **System impact:** Failure in the initial ingestion stage (Reddit Producer) propagated through the entire chain (Sentiment Stream → Data Sync), resulting in empty dashboards and non-functional analytical routes.

4.3.2 Proposed Approaches

- **Approach 1:** Deploying individual microservices independently and troubleshooting API connectivity errors reactively during runtime.
- **Approach 2:** Implementing a structured deployment plan that treats external API credentials as mandatory environmental prerequisites.
- **Trade-offs of each approach:** Approach 1 minimizes setup time but risks cascading downtime and difficult-to-trace "silent" failures. Approach 2 increases preparation effort but ensures system-wide stability upon initialization.

4.3.3 Future Recommendations

- **Technical insights:** External API dependencies are the most fragile points in a data pipeline and must be isolated and validated before triggering internal processing logic.
- **Best practices:** Use a "fail-fast" mechanism where services verify credential access during the bootstrap phase rather than waiting for the first data request.
- **Strategic Advice:** Maintain a formal pre-deployment checklist covering API quota limits, credential expiration, and network access to external endpoints to ensure complete system functionality.